

SQLP 과목 III 튜닝 스타일 모의문제 · 5회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

Multiblock I/O가 한 번의 I/O Call로 읽어 들이는 블록 수, 그리고 그 반대편에 놓인 Single Block I/O·Random 액세스에 대한 설명으로 가장 적절하지 않은 것은?

- ① 한 번의 Multiblock I/O로 읽는 블록 수는 DB_FILE_MULTIBLOCK_READ_COUNT에 지정한 값과 같게 유지되며, 익스텐트 경계를 넘어가는지나 그 안에 이미 버퍼 캐시에 올라온 블록이 섞여 있는지는 이 개수를 바꾸지 못한다.
- ② Full Table Scan은 세그먼트의 블록을 물리적 저장 순서대로 훑어 나가는 Sequential 액세스이며, 인접한 여러 블록을 한 번에 읽어 들이는 Multiblock I/O가 동반되어 대기 이벤트는 db file scattered read로 나타난다.
- ③ 인덱스에서 얻은 ROWID로 테이블의 특정 블록 하나를 임의 위치에서 집어 오는 Random 액세스는 Single Block I/O로 처리되며, 그 대기 이벤트는 이름과 달리 db file sequential read이다.
- ④ DB_FILE_MULTIBLOCK_READ_COUNT를 크게 잡으면 같은 양의 블록을 훑을 때 한 Call에 묶이는 블록 수가 늘어 I/O Call 횟수가 줄어드는 경향이 있어, 넓은 범위를 훑는 Full Table Scan에 유리하게 작용할 수 있다.

2

버퍼 캐시를 용도별로 나누는 KEEP·RECYCLE·DEFAULT 버퍼 풀과 그 관리 방식에 대한 설명으로 가장 적절한 것은?

- ① KEEP 버퍼 풀에 배정된 블록은 한 번 적재되면 LRU 교체 대상에서 완전히 빠져, 인스턴스가 살아 있는 동안 캐시에서 밀려나지 않고 그대로 상주한다.
- ② RECYCLE 버퍼 풀은 블록을 올리는 즉시 사용 직후 버리도록 되어 있어, 같은 블록을 곧바로 다시 참조하더라도 캐시에 남아 있지 않아 디스크에서 새로 읽어 들인다.
- ③ 어떤 세그먼트의 블록을 KEEP·RECYCLE·DEFAULT 중 어느 풀에 올릴지는 옵티마이저가 실행 시점에 그 블록의 접근 빈도를 관찰해 자동으로 배정한다.
- ④ 세 버퍼 풀은 하나의 버퍼 캐시를 용도에 따라 나눈 영역으로 각각 독립된 LRU 리스트로 관리되며, 어떤 세그먼트의 블록을 어느 풀에 캐싱할지는 그 세그먼트의 BUFFER_POOL 저장 속성으로 지정한다.

전자결재 시스템의 야간 배치가 결재 대기 문서 약 8,000건을 처리하면서, 문서마다 문서번호를 문자열 리터럴로 이어붙여(WHERE 문서번호 = 'DOC-2026-...') SQL을 새로 만들어 실행한다. 배치가 38초를 넘겨 느리다는 신고가 들어와 TKPROF를 수집했더니 아래처럼

비재귀(non-recursive)·재귀(recursive) 통계가 분리되어 출력됐다. 이 트레이스에 대한 옳은 진단은? (단, 옵티마이저 통계는 최신이고 결재선·문서 테이블의 인덱스는 정상이며, 배치는 문서 1건당 리터럴 SQL 1종을 생성한다.)

아 래							
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	8000	27.000	31.000	0	0	0	0
Execute	8000	3.200	3.400	0	64000	8000	8000
Fetch	8000	1.800	4.100	400	96000	0	8000
Total	24000	32.000	38.500	400	160000	8000	16000
OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	16000	6.000	6.200	0	0	0	0
Execute	48000	5.400	5.600	120	240000	4000	120000
Fetch	96000	4.600	4.900	800	320000	0	400000
Total	160000	16.000	16.700	920	560000	4000	520000
Misses in library cache during parse: 8000							

- ① 재귀 Execute가 48,000회로 비재귀 Execute 8,000회의 6배이고 재귀 Rows가 520,000행에 이르므로, 배치가 문서 한 건마다 상관 서브쿼리를 여러 번 돌려 행을 퍼올리는 데이터 처리 로직이 진짜 병목이다. 비재귀 Parse CPU 27초는 그 무거운 SQL의 계획을 세우느라 든 부수 비용이니, 서브쿼리를 조인으로 재작성해 재귀 Query 560,000 블록을 걷어내면 38.5초가 해소된다.
- ② 재귀 Fetch Disk 800을 포함해 재귀 Disk가 920으로 비재귀 Disk 400의 2배가 넘고 재귀 Query도 560,000 블록에 달하니, 파싱이 아니라 데이터 디크셔너리 블록을 매번 물리적으로 다시 읽는 것이 정체 원인이다. 버퍼 캐시를 키워 재귀 블록 I/O를 메모리에 상주시키면 재귀 Elapsed 16.7초가 줄어 배치 38.5초도 함께 짧아진다.
- ③ 비재귀 Parse CPU 27초는 옵티마이저가 문서 8,000건 각각에 맞는 실행계획을 정교하게 최적화하느라 쓴 정상 비용이고, 그 덕에 비재귀 Execute CPU 3.2초·Fetch CPU 1.8초로 실행이 억제되었다. 재귀 Call 160,000회 역시 그 좋은 계획을 실행하며 디크셔너리를 참조한 결과이므로, Elapsed 38.5초는 8,000건 처리의 합당한 값이고 현재 상태가 이미 최적이다.
- ④ 비재귀 Parse CPU 27초가 가리키듯 리터럴 SQL의 하드파싱이 주범이고, 재귀 Parse 16,000·재귀 Call 160,000은 그 하드파싱이 파싱 중 데이터 디크셔너리를 조회하며 파생시킨 파싱 부하이지 애플리케이션 데이터 로직이 무거워서가 아니다. 바인드 변수로 하드파싱을 없애면 재귀 디크셔너리 호출도 함께 줄어든다.

4

EXPLAIN PLAN으로 적재한 예상 실행계획을 DBMS_XPLAN.DISPLAY로 조회할 때의 format 옵션과 계획에 표시되는 행 수(카디널리티)의 성격에 대한 설명으로 가장 적절한 것은?

- ① DISPLAY가 각 단계에 보여 주는 행 수는 옵티마이저가 통계를 근거로 산출한 예상 카디널리티이며, 통계가 낡았거나 조건 컬럼 간 상관관계가 있으면 실제 수행 시 반환되는 행 수와 크게 어긋날 수 있다.
- ② format 인자를 'ALL'이나 'ADVANCED'로 높이면 각 단계에서 실제로 반환된 행 수까지 함께 표시되므로, 예상 카디널리티와 실제 행 수를 한 화면에서 나란히 비교할 수 있다.
- ③ format을 지정하지 않으면 기본값이 'BASIC'이라 카디널리티와 Cost 열이 아예 출력되지 않으므로, 추정 행 수를 보려면 format을 반드시 'TYPICAL' 이상으로 올려 지정해야 한다.
- ④ DISPLAY 출력 하단 Note에 동적 샘플링(dynamic statistics)이 사용됐다는 문구가 찍히면, 그 계획은 문장을 실제로 수행해 얻은 실제 계획이므로 예상 계획이 아니라 실제 실행계획으로 보아야 한다.

5

항만 물류 시스템에서 컨테이너반출입 테이블(약 9,000만 건)과 선박 테이블(약 4만 건)을 조인해 선박별 반출입 건수를 집계하는 SQL의 TKPROF 결과이다. 아래 트레이스에 대한 해석으로 가장 적절한 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.006	0.006	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	501	8.994	71.994	200000	2500000	0	5000
Total	503	9.000	72.000	200000	2500000	0	5000
Rows	Row Source Operation						
5000	HASH GROUP BY (cr=2500000 pr=200000 pw=0 time=71900000 us)						
90000000	HASH JOIN (cr=2500000 pr=200000 pw=0 time=70500000 us)						
40000	TABLE ACCESS FULL 선박 (cr=140000 pr=9000 pw=0 time=1800000 us)						
90000000	TABLE ACCESS FULL 컨테이너반출입 (cr=2360000 pr=191000 pw=0 time=66500000 us)						

- ① Fetch 501회가 CPU 8.994초를 거의 다 쓰며 총 CPU 9초를 쌓았고, HASH JOIN이 9,000만 행을 만들어 내며 time=70.5초를, HASH GROUP BY가 그 위에서 time=71.9초를 소비했으므로 이 SQL은 조인·집계 연산에 CPU를 갈아 넣는 CPU 바운드다. 병렬도(DOP)를 높여 9,000만 행의 해시 처리를 여러 프로세스에 분산하는 것이 Elapsed 72초를 줄이는 최우선 처방이다.
- ② Row Source 네 노드 어디에도 INDEX 스캔이 없고 4만 건짜리 선박마저 TABLE ACCESS FULL로 cr=140,000·pr=9,000·time=1.8초를 쓰고 있으니, 조인 키에 인덱스가 없어 소량 테이블까지 통째로 훑는 것이 병목이다. 선박에 인덱스를 만들어 HASH JOIN을 인덱스 경유 조인으로 바꾸면 Fetch Elapsed 71.994초가 해소된다.
- ③ 논리 I/O 250만 블록 중 20만(약 8%)을 디스크에서 읽어 BCHR은 약 92%이며, CPU 9초는 Elapsed 72초의 약 12.5%뿐이라 나머지 약 63초가 대기다. cr·pr·time이 모두 컨테이너반출입 Full Scan(cr=2,360,000, pr=191,000, time≈66.5초)에 집중되므로 병목은 9,000만 건 컨테이너반출입 테이블 액세스다.
- ④ Disk 200,000에 BCHR 92%이고 HASH JOIN이 9,000만 행을 다루며 time=70.5초를 쓴 것으로 보아, 해시 영역이 부족해 조인 중간 결과가 임시 세그먼트로 스푼한 것이 원인이다. 스푼한 파티션을 되읽느라 pr 200,000이 발생했으므로, PGA를 키워 스푼을 없애면 Elapsed 72초에서 CPU 9초를 뺀 약 63초의 대기가 사라진다.

차량정비 이력 테이블(약 540만 건)에는 결합 인덱스 정비_IX = (정비소코드, 접수일자, 정비구분)가 있다. 정비소명을 함께 얻기 위해 정비소 테이블(기본키 정비소_PK = 정비소코드)과 조인하는 다음 SQL의 실행계획이다.

```
SELECT r.정비번호, r.접수일자, r.정비구분, s.정비소명
FROM 정비이력 r, 정비소 s
WHERE r.정비소코드 = 'C07'
AND r.접수일자 >= DATE '2026-06-01'
AND r.접수일자 < DATE '2026-07-01'
AND r.정비구분 = '엔진'
AND r.정비상태 = '완료'
AND s.정비소코드 = r.정비소코드;
```

이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래					
Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(268	2100	113400)
1	0	NESTED LOOPS	(268	2100	113400)
2	1	TABLE ACCESS BY INDEX ROWID 정비이력	(263	2100	75600)
3	2	INDEX RANGE SCAN 정비_IX	(19	8400	)
4	1	TABLE ACCESS BY INDEX ROWID 정비소	(1	1	18)
5	4	INDEX UNIQUE SCAN 정비소_PK	(0	1	)

- ① 정비_IX의 정비소코드(등치)·접수일자(범위)·정비구분(등치) 세 칼럼이 모두 인덱스 액세스 조건이 되어 INDEX RANGE SCAN(Id 3)의 스캔 시작·종료 지점을 함께 좁히며, 그 결과가 Id 3의 Card 8,400이다. 인덱스에 없는 정비상태만 Id 2 테이블 액세스의 필터로 남아 Card가 2,100으로 준다.
- ② 정비상태='완료'는 정비_IX가 훑는 리프 엔트리마다 확인되는 인덱스 필터라서 INDEX RANGE SCAN(Id 3, Cost 19) 단계에서 이미 걸러지고, 그 결과가 Id 3의 Card 8,400이다. Id 2의 Card 2,100은 그 뒤 테이블 블록을 읽으며 접수일자 6월 범위를 다시 확인해 줄어든 값이다.
- ③ 선두 칼럼 정비소코드(등치)와 접수일자(범위)까지가 액세스 조건으로 스캔 범위를 정하고, 범위 칼럼 뒤에 놓인 정비구분은 스캔 범위를 좁히지 못하는 인덱스 필터로, 인덱스에 없는 정비상태는 Id 2 테이블 액세스 단계의 필터로 처리된다.
- ④ INDEX RANGE SCAN(Id 3)이 돌려준 Card 8,400이 곧 정비상태 조건까지 반영된 이 SQL의 최종 결과 건수이며, Id 2 이후 단계에서 건수 감소는 일어나지 않는다. Id 1 NESTED LOOPS의 Card 2,100은 정비소_PK가 1:1로 매칭돼 나온 조인 결과를 따로 표기한 값이고, Cost 268도 그에 맞춰 산정되었다.

7

설비 점검 시스템의 점검이력 테이블(약 1,000만 건, 세그먼트 약 100,000 블록)에서 점검이력_IX 인덱스를 경유해 특정 기간의 점검 내역을 조회한 SQL의 트레이스이다. 이 트레이스에 대한 옳은 진단은? (단, 옵티마이저 통계는 최신이고 인덱스는 Unusable 상태가 아니며, 이 SQL의 SELECT 목록은 점검일자, 점검결과 두 컬럼뿐이다.)

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.005	0.005	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	3001	5.395	37.995	60000	271800	0	600000
Total	3003	5.400	38.000	60000	271800	0	600000
Rows	Row Source Operation						
600000	TABLE ACCESS BY INDEX ROWID 점검이력 (cr=271800 pr=60000 pw=0 time=37600000 us)						
600000	INDEX RANGE SCAN 점검이력_IX (cr=1800 pr=300 pw=0 time=620000 us)						

- ① Disk 60,000이 Query 271,800의 약 22%로 BCRH이 약 78%에 그치고, Elapsed 38.0초에서 CPU 5.395초를 뺀 약 32.6초가 물리 읽기 대기이므로 버퍼 캐시 부족이 근본 원인이다. 인덱스는 cr=1,800·pr=300으로 이미 캐시에 잘 얹혀 있고 테이블 액세스 노드만 pr=60,000을 겪었으니, 캐시를 키워 점검이력 10만 블록을 상주시키면 이 SQL은 해결된다.
- ② 테이블 액세스 cr 271,800이 세그먼트 블록 수 100,000을 두 배 넘게 웃돌아 인덱스 손익분기점을 이미 넘겼으므로 Full Scan 전환만이 유일한 해법이다. 점검이력이 1,000만 건·10만 블록으로 커서 인덱스를 더 얹어 봐야 리프가 두꺼워질 뿐, 커버링 인덱스로는 이 38초를 개선할 수 없다.
- ③ 테이블 랜덤 블록 270,000을 ROWID 600,000으로 나눈 값이 약 0.45로 1보다 작으니 한 블록에서 여러 행을 뽑아 온 셈이라 클러스터링 팩터는 오히려 우수하다. 따라서 38초 대기의 정체는 테이블이 아니라 INDEX RANGE SCAN(cr=1,800, time=0.62초)이 겪은 인덱스 리프 블록 분할이다.
- ④ 테이블 랜덤 액세스가 ROWID 60만 건에 블록 I/O 27만을 써 CF가 테이블 블록 수(100,000)의 약 45배로 나뉘고, 그 랜덤 블록이 cr의 대부분이다. SELECT가 두 컬럼뿐이라 커버링 인덱스로 TABLE ACCESS BY INDEX ROWID를 없애면 cr이 인덱스 1,800에서 리프가 넓어져도 수천 블록대라 Full Scan(100,000)보다 유리하다.

8

리프 레벨에 행 데이터를 정렬해 담은 클러스터형 인덱스(SQL Server)·IOT(오라클)와 리프에 키+행 위치만 담은 일반 B-Tree 인덱스, 그리고 Index Unique Scan에 대한 설명으로 가장 적절한 것은?

- ① SQL Server의 비클러스터형(nonclustered) 인덱스는 리프 블록에 행 데이터 전체를 인덱스 키 순서로 담고 있어, 그 인덱스만 읽으면 어떤 컬럼을 조회하든 원본 테이블을 되짚지 않고 결과를 낼 수 있다.
- ② 클러스터형 인덱스는 리프가 곧 데이터이므로 한 테이블에 여러 개를 만들어 두면 조회마다 서로 다른 정렬본을 골라 써서, 어떤 검색 조건이 들어와도 테이블 액세스 없이 커버링으로 처리된다.
- ③ 오라클의 IOT나 SQL Server의 클러스터형 인덱스는 리프 레벨에 행 데이터를 인덱스 키 순서로 저장하므로, 키로 조회하면 리프에 도달하는 것만으로 행을 얻어 힙 테이블을 되짚는 ROWID/RID 액세스가 필요 없다.
- ④ 유일(UNIQUE) 인덱스에 등가 조건을 주어 일어나는 Index Unique Scan은, 조건을 만족하는 행이 여러 건이더라도 리프 블록을 오른쪽으로 계속 이어 읽어 해당 행을 모두 훑은 뒤 종료한다.

결합 인덱스의 컬럼 구성·순서와 컬럼 추가에 따른 효과·부하에 대한 설명으로 가장 적절하지 않은 것은?

- ① 결합 인덱스의 선행 컬럼에는 조회에서 등가(=) 조건으로 자주 쓰이면서 변별력이 높은 컬럼을 두는 편이 유리하며, 그래야 스캔 시작·종료 지점을 좁게 잡아 훑는 리프 블록 수를 줄일 수 있다.
- ② 인덱스에 컬럼을 하나 더하면 리프 엔트리 길이가 늘어 인덱스가 커지고, 그 테이블에 INSERT·UPDATE·DELETE가 일어날 때마다 갱신·정렬해야 할 인덱스 데이터가 많아져 DML 부하가 증가한다.
- ③ 같은 컬럼 집합이라도 조회 패턴에 따라 컬럼 순서를 달리 설계해야 하며, 여러 쿼리에서 등가 조건으로 자주 등장하는 컬럼을 앞쪽에 배치하는 편이 스캔 범위를 좁히는 데 유리하다.
- ④ 결합 인덱스에 컬럼을 추가하면 그 컬럼이 어느 위치에 오든, 또 앞선 컬럼에 어떤 조건이 걸리든, 조건절에 등장하기만 하면 인덱스 스캔의 시작·종료 지점을 좁혀 액세스 범위를 줄여 준다.

수강신청 테이블(약 4,100만 건)과 강좌 테이블(약 12만 건)을 조인하는 SQL의 실행계획이다.

수강신청에는 수강_IX = (개설학기, 학과코드, 신청일시)가, 강좌에는 기본키 강좌_PK = (강좌ID)가 있다. 개설학기 = '2026-2'이면서 학과코드 = 'CSE'인 수강신청은 약 530건이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

```
SELECT e.신청번호, e.신청일시, c.강좌명, c.담당교수
FROM 수강신청 e, 강좌 c
WHERE e.개설학기 = '2026-2'
      AND e.학과코드 = 'CSE'
      AND c.강좌ID = e.강좌ID;
```

Id	Operation	Name
0	SELECT STATEMENT	
1	NESTED LOOPS	
2	TABLE ACCESS BY INDEX ROWID	수강신청
* 3	INDEX RANGE SCAN	수강_IX
4	TABLE ACCESS BY INDEX ROWID	강좌
* 5	INDEX UNIQUE SCAN	강좌_PK

- ① 이 NESTED LOOPS에서 반복 탐색 부담은 인너 강좌(약 12만 건)의 크기가 좌우하고, 드라이빙 수강신청은 수강_IX(Id 3)로 이미 약 530건까지 줄어 더 손덜 여지가 없다. 따라서 강좌 테이블을 줄이거나 강좌_PK INDEX UNIQUE SCAN(Id 5)을 재구성해 인너 탐색 비용을 낮추는 것이 반복 부담을 더는 핵심 처방이다.
- ② 드라이빙(선행)은 먼저 나온 수강신청(Id 2)이고, 수강_IX(Id 3)로 좁힌 약 530건 각각에 대해 강좌_PK INDEX UNIQUE SCAN(Id 5)이 반복 수행된다. NL은 드라이빙 앞쪽부터 조인해 내려가므로 상위 일부만 필요할 때 첫 행을 빠르게 내는 부분범위처리에 유리하다.
- ③ Id 4 TABLE ACCESS BY INDEX ROWID 강좌가 인너를 한 번에 읽어 강좌ID로 정렬·적재한 뒤 Id 2의 수강신청 약 530건과 병합하므로, 수강신청 4,100만 건과 강좌 12만 건은 각각 한 번씩만 스캔되고 Id 5 강좌_PK INDEX UNIQUE SCAN도 한 차례만 수행된다. 반복이 없으니 인너가 커져도 조인 비용은 늘지 않는다.
- ④ 이 플랜의 드라이빙은 Id 4의 강좌이고 수강신청을 인너로 두어 수강_IX(Id 3)를 강좌 12만 건마다 반복 탐색하므로, 부분범위처리 이득은 인너 수강신청에서 뽑히는 건수가 많을수록 커진다. 개설학기='2026-2' AND 학과코드='CSE'인 약 530건은 그 반복 탐색으로 누적된 결과다.

11

정렬된 상위 N건만 뽑는 Top-N 쿼리(ROWNUM <= N 또는 FETCH FIRST N ROWS ONLY)와 부분범위 처리에 대한 설명으로 가장 적절한 것은?

- ① ROWNUM <= N을 걸면 옵티마이저는 언제나 결과 집합 전체를 정렬로 완성한 뒤 앞 N건을 잘라 내므로, 정렬 순서를 제공하는 인덱스가 있어도 정렬 단계는 사라지지 않는다.
- ② 정렬 순서를 그대로 내주는 인덱스가 있으면 ROWNUM <= N은 인덱스를 앞에서부터 훑다가 N번째 행을 채운 시점에 스캔을 멈추므로(부분범위 처리), 별도 정렬 오퍼레이션 없이 상위 N건만 읽고 끝난다.
- ③ ORDER BY 없이 ROWNUM <= N만 쓰면 먼저 조회된 아무 N건이 담기는데, 인덱스가 있으면 이때도 값 순으로 정렬된 상위 N건(Top-N)이 저절로 보장된다.
- ④ NL 조인 뒤에 ROWNUM <= N을 두면 필요한 N건을 다 채운 시점에 조인을 멈출 수 없고, 조인 대상 집합을 끝까지 소진한 뒤에야 앞 N건을 남기므로 inner 반복 횟수를 줄이는 효과가 없다.

12

오라클 옵티마이저의 두 방식, RBO(규칙 기반)와 CBO(비용 기반)에 대한 설명으로 가장 적절하지 않은 것은?

- ① RBO는 미리 정해진 액세스 경로 우선순위 규칙(약 15가지 순위)에 따라 실행계획을 결정하며, 데이터 건수나 분포 같은 통계는 판단에 쓰지 않는다.
- ② RBO에서는 유일 인덱스 등치 액세스가 전체 테이블 스캔보다 높은 순위를 가지므로, 실제 데이터가 적어 전체 스캔이 더 빠른 상황에서도 규칙상 인덱스 경로가 선택된다.
- ③ CBO는 옵티마이저 통계(카디널리티·선택도 등)를 바탕으로 각 후보 계획의 예상 비용을 산정해 가장 낮은 것을 고르므로, 통계가 부정확하면 계획 품질이 나빠질 수 있다.
- ④ CBO도 내부적으로는 RBO의 규칙 우선순위표를 먼저 적용해 후보 계획의 순위를 정한 뒤 통계로 그 순위를 재확인하므로, 두 방식은 같은 규칙 축을 공유한다.

13

구매발주 조회 화면은 발주상태코드 값만 바꿔 가며 리터럴을 SQL 텍스트에 직접 박아 넣는 방식으로 질의를 던진다. 세션에 CURSOR_SHARING = FORCE를 걸고 상태코드 'S1'·'S2'·'S3'로 세 번 조회한 뒤 라이브러리 캐시(v\$sql)를 요약한 결과가 아래와 같다. CURSOR_SHARING과 커서 공유에 대한 설명으로 가장 적절하지 않은 것은?

아 래

-- 세션 기본: ALTER SESSION SET CURSOR_SHARING = FORCE;

select 품목명 from 발주 where 발주상태코드 = 'S1';

select 품목명 from 발주 where 발주상태코드 = 'S2';

select 품목명 from 발주 where 발주상태코드 = 'S3';

[라이브러리 캐시(v\$sql) 요약 - CURSOR_SHARING=FORCE]

SQL_ID	SQL_TEXT (전두 요약)	PARSE_CALLS	EXECUTIONS	LOADS
h7k3m9r2t5wqz	select 품목명 from 발주 where 발주상태코드 = : "SYS_B_0"	3	3	1

- ① CURSOR_SHARING=FORCE는 SQL 텍스트의 리터럴('S1'·'S2'·'S3')을 시스템 생성 바인드 변수(: "SYS_B_0")로 치환하므로, 값만 다른 세 SQL이 하나의 부모 커서(h7k3...)를 공유한다.
- ② CURSOR_SHARING=FORCE는 리터럴뿐 아니라 키워드 대소문자·공백·힌트까지 정규화하므로, 대소문자나 공백만 달라진 SQL도 자동으로 같은 커서로 합쳐진다.
- ③ 리터럴을 바인드로 치환해 공유하면 하드파싱(LOADS)은 한 번으로 줄지만, 값 분포가 편중된 칼럼에서는 첫 실행의 바인드 값에 맞춰진 계획을 이후 값들이 그대로 쓰는 바인드 변수 엿보기(bind peeking)의 부작용이 생길 수 있다.
- ④ 기본값 EXACT였다면 'S1'·'S2'·'S3' 세 리터럴이 각각 다른 SQL_ID로 하드파싱되지만, FORCE에서는 v\$sql에 : "SYS_B_0"를 담은 커서 하나만 남고 EXECUTIONS가 3으로 누적된다.

상관 서브쿼리에 대한 옵티마이저의 서브쿼리 Unnesting과 조건절 이행에 대한 설명으로 가장 적절한 것은?

- ① EXISTS로 건 상관 서브쿼리를 옵티마이저가 Unnesting하면 세미(semi) 조인으로 풀리므로, 행마다 서브쿼리를 반복 수행하는 필터 방식 대신 조인 방식(NL·해시·소트머지)과 순서를 비용으로 골라 처리할 수 있게 된다.
- ② NOT EXISTS로 건 상관 서브쿼리는 짝이 하나도 없음을 확인해야 끝나는 부정 조건이라 조인으로 승격할 수 없어 Unnesting 대상이 되지 않고, 바깥 행마다 서브쿼리를 재수행하는 필터 형태로만 처리된다. 그래서 EXISTS와 달리 조인 방식·순서 선택이 열리지 않는다.
- ③ 조건절 이행(Transitive Predicate Generation)은 Unnesting이 서브쿼리를 조인으로 바꾼 뒤 그 조인 조건을 바깥 집합으로 전파하는 마지막 세부 단계로, 둘은 한 절차를 앞뒤로 나눠 부른 이름이다. 그래서 Unnesting이 없으면 조건절 이행도 적용되지 않는다.
- ④ SELECT 절에서 값을 하나 돌려주는 스칼라 서브쿼리는 바깥 행마다 수행되는 대표적인 상관 서브쿼리이므로, 옵티마이저는 이를 세미 조인으로 Unnesting해 중복 증식 없이 값을 붙인다. WHERE 절의 EXISTS·IN은 그 특수한 변형이다.

상여지급 마감 배치는 사원별 성과 집계를 상여 테이블에 반영한다. 아래 (가)는 수정 가능 조인 뷰(updatable join view)를 통한 UPDATE이고, (나)는 같은 목적을 MERGE로 처리한 문장이다. 사원.사원번호는 사원 테이블의 PK이며 상여.사원번호는 사원을 참조한다. 두 문장의 동작과 조건에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
-- (가) 수정 가능 조인 뷰를 통한 UPDATE
UPDATE (
    SELECT b.상여금액, s.기준급
    FROM   상여 b, 사원 s
    WHERE  b.사원번호 = s.사원번호
)
SET 상여금액 = 기준급 * 0.15;

-- (나) 같은 반영을 MERGE로
MERGE INTO 상여 t
USING 성과집계 s
ON (t.사원번호 = s.사원번호)
WHEN MATCHED THEN
    UPDATE SET t.상여금액 = s.산정액
WHEN NOT MATCHED THEN
    INSERT (사원번호, 상여금액) VALUES (s.사원번호, s.산정액);
```

- ① (가)에서 UPDATE 대상 칼럼(상여금액)이 속한 상여는 조인에서 키가 보존되는(key-preserved) 테이블이어야 한다. 사원.사원번호가 PK라 상여 한 행에 사원 한 행만 대응하므로, 상여가 key-preserved가 되어 상여금액 수정이 허용된다.
- ② (나)의 MERGE는 한 문장으로 대상(t)에 매칭 행이 있으면 UPDATE, 없으면 INSERT를 수행하며, 매칭 여부는 ON 절 조건(t.사원번호 = s.사원번호)으로 판정한다.
- ③ (가)의 조인 뷰에서는 key-preserved가 아닌 사원의 칼럼(기준급)도 조인에 참여했으므로 함께 UPDATE할 수 있고, (나)의 MERGE는 WHEN MATCHED와 WHEN NOT MATCHED 두 절을 반드시 함께 두어야만 실행된다.
- ④ (가)가 성립하려면 조인 후에도 상여의 사원번호가 유일해야 한다. 만약 사원 쪽 사원번호가 유일하지 않으면 상여 한 행이 여러 사원 행과 겹쳐 어떤 값으로 갱신할지 모호해지므로, 조인 뷰 UPDATE가 거부된다.

민원 접수 테이블은 접수지역코드(2자리 VARCHAR2)를 기준으로 아래처럼 LIST 파티셔닝되어 있다. 파티셔닝의 목적과 Partition Pruning의 작동에 대한 설명으로 가장 적절하지 않은 것은?

아 래

```
CREATE TABLE 민원 (
  민원번호      NUMBER,
  접수지역코드  VARCHAR2(2),
  접수일자      DATE,
  처리상태      VARCHAR2(10)
)
PARTITION BY LIST (접수지역코드) (
  PARTITION p_seoul    VALUES ('11'),
  PARTITION p_busan    VALUES ('26'),
  PARTITION p_incheon  VALUES ('28'),
  PARTITION p_gyeonggi VALUES ('41'),
  PARTITION p_etc      VALUES (DEFAULT)
);
```

- ① WHERE SUBSTR(접수지역코드, 1, 2) = '11' 조건은 접수지역코드의 앞 두 자리를 그대로 뽑은 것이라 값이 코드 자체와 같으므로, 옵티마이저가 p_seoul 한 파티션으로 Pruning한다.
- ② WHERE 접수지역코드 = '26' 조건은 파티션 키를 가공하지 않았으므로, 옵티마이저가 값을 파티션 목록과 대조해 p_busan 한 파티션만 접근한다.
- ③ 파티셔닝의 목적은 성능(Pruning으로 스캔량 축소)에 국한되지 않는다. 파티션 단위 백업·DROP PARTITION 같은 관리성과, 한 파티션의 장애·작업이 다른 파티션 가용성에 미치는 영향을 줄이는 가용성도 포함된다.
- ④ WHERE 접수지역코드 IN ('11','41') 조건은 두 리터럴이 각각 파티션 목록과 대조돼 p_seoul·p_gyeonggi 두 파티션만 읽고 나머지는 건너뛰므로 Pruning이 작동한다.

화물추적 배치에서 운송기록 테이블(약 1억 9천만 건)과 검수내역 테이블(약 7,200만 건)을 운송장번호로 병렬 해시 조인한다. 운송기록은 운송장번호 기준 해시 16 파티션으로 나뉘어 있으나, 검수내역은 파티셔닝돼 있지 않다. 이 병렬 실행계획은 두 테이블을 병렬 서버(Slave) 사이에 어떻게 분배해 조인하는가? 가장 정확히 지목한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10001			Q1,01	P->S
3	HASH JOIN				Q1,01	PCWP
4	PX PARTITION HASH ALL		1	16	Q1,01	PCWC
5	TABLE ACCESS FULL	운송기록	1	16	Q1,01	PCWP
6	PX RECEIVE				Q1,01	PCWP
7	PX SEND PARTITION (KEY)	:TQ10000			Q1,00	P->P
8	PX BLOCK ITERATOR				Q1,00	PCWC
9	TABLE ACCESS FULL	검수내역			Q1,00	PCWP

- ① 운송기록과 검수내역이 둘 다 운송장번호로 해시 파티셔닝돼 있어, Id 4 PX PARTITION HASH ALL(Pstart 1~Pstop 16)이 대응 파티션 쌍을 한 서버에 모아 준다. Id 3 HASH JOIN은 재분배 없이 쌍만 조인하는 (full) 파티션 와이즈 조인이고, Id 7 :TQ10000은 결과를 QC로 넘기는 통로다.
- ② 운송기록은 파티션 단위로 각 서버가 자기 파티션을 재분배 없이 읽고(Id 4 PX PARTITION HASH ALL), 검수내역만 Id 7 PX SEND PARTITION (KEY)로 운송기록의 파티션 경계에 맞춰 재분배하는 부분(partial) 파티션 와이즈 조인이다.
- ③ Id 7의 :TQ10000과 Id 2의 :TQ10001 두 테이블 큐를 거쳐 운송기록과 검수내역을 양쪽 다 운송장번호 해시로 재분배하는 hash-hash 분배이며, Id 5·Id 9 위에 각각 PX SEND HASH가 놓여 두 스트림을 Q1,00·Q1,01의 새 해시 버킷으로 옮긴 뒤 Id 3 HASH JOIN이 받아 조인한다.
- ④ 검수내역은 Id 8 PX BLOCK ITERATOR로 각 서버가 자기 블록 범위를 재분배 없이 읽고, 파티셔닝된 운송기록만 Id 7 PX SEND PARTITION (KEY)로 검수내역이 옮겨진 경계에 맞춰 Q1,00에서 Q1,01로 재분배하는 부분(partial) 파티션 와이즈 조인이다.

SELECT 절에서 호출하는 사용자 정의 함수(PL/SQL function)의 비효율과 그 대체 방안에 대한 설명으로 가장 적절한 것은?

- ① SELECT 절에서 호출한 PL/SQL 함수는 커서를 열 때 함께 로드되어 SQL 실행 엔진 안에서 그대로 수행되므로 SQL↔PL/SQL 경계를 넘는 컨텍스트 스위치가 일어나지 않는다. 따라서 결과 행이 늘어 함수 호출이 반복돼도 늘어나는 것은 함수 본문의 순수 연산량뿐이고, 스위치로 인한 부하는 생기지 않는다.
- ② SELECT 절의 사용자 정의 함수는 결과 행마다 호출되어 그때마다 SQL↔PL/SQL 컨텍스트 스위치가 일어나고, 함수 본문이 다시 SQL을 수행하면 재귀 호출까지 겹쳐 행 수가 많을수록 부하가 커지므로, 조인이나 스칼라 서브쿼리로 대체해 호출 자체를 줄이는 것이 효과적이다.
- ③ 컨텍스트 스위치는 함수 본문의 SQL이 재귀 호출로 다시 SQL 엔진에 진입할 때 생기는 비용이므로, 본문에서 SQL을 돌리지 않는 순수 계산 함수라면 결과 행마다 호출되어도 컨텍스트 스위치 부하는 발생하지 않는다. 그런 함수는 조인이나 스칼라 서브쿼리로 대체할 실익도 없다.
- ④ 함수 호출을 스칼라 서브쿼리로 감싸면 오라클이 입력값과 결과를 해시 캐시에 적재하는 스칼라 서브쿼리 캐싱이 적용되고, 이 캐시는 입력값의 종류와 무관하게 동작하므로 입력이 행마다 제각각이더라도 호출은 한 번으로 줄고 SQL↔PL/SQL 컨텍스트 스위치도 사라진다.

유량계측 시스템에서 유량계 'FM-720'의 누적유량 행(초기 4200)을 TX1이 한 트랜잭션 안에서 두 번 읽는데, 그 사이 TX2가 값을 4650으로 갱신하고 커밋한다(아래 타임라인). TX1을 SQL Server의 SNAPSHOT 격리수준으로 실행할 때, TX1의 두 번째 읽기 값과 그 해석으로 가장 적절한 것은? (단, TX2는 기본 READ COMMITTED이며, 비교를 위해 동일 시나리오를 다른 읽기 일관성 방식에서도 함께 따진다.)

- ① SNAPSHOT 격리의 TX1은 트랜잭션 시작 시점 스냅샷을 읽으므로 (A)=(B)=4200으로 반복 가능하다. 만약 오라클 기본(문장수준 읽기 일관성)이나 SQL Server 기본 READ COMMITTED였다면 (B)=4650이 되어 두 읽기가 달라진다.
- ② SNAPSHOT 격리의 TX1은 t2에서 (A)=4200을 읽지만, t3에 TX2가 4650으로 UPDATE하고 커밋한 뒤 t4의 두 번째 SELECT는 새 문장이라 커밋된 최신값 (B)=4650을 읽는다. 따라서 SNAPSHOT은 두 읽기 값이 갈리는 비반복 읽기(non-repeatable read)를 그대로 허용한다.
- ③ RCSI(READ_COMMITTED_SNAPSHOT)를 켜면 TX1의 t4 SELECT도 t1 BEGIN TRAN 시점의 행 버전을 보게 되어 (A)=(B)=4200이 되므로, t3의 4650 커밋을 무시한다는 점에서 SNAPSHOT 격리와 결과가 같아진다. 두 옵션의 차이는 스냅샷 기준이 아니라 버전 저장소 사용 여부뿐이다.
- ④ 오라클 MVCC는 엔드 세그먼트로 읽기 일관성을 보장하되 그 기준이 문장이 아니라 트랜잭션 시작 시점이므로, TX1이 t4에서 읽는 (B)도 t1 시점 버전인 4200이 되어 SQL Server SNAPSHOT과 값이 같아진다. t3에 커밋된 4650은 TX1이 COMMIT하는 t5 이후에야 보인다.

입찰 마감 작업에서 세션 41이 입찰 테이블 전체에 LOCK TABLE 입찰 IN SHARE MODE를 건 상태다. 그 사이 세션 55가 특정 입찰 건의 낙찰가를 UPDATE하려 한다. 이때 v\$lock을 조회한 결과가 아래와 같다. 이 테이블 수준 락(TM) 상태에 대한 해석으로 가장 적절하지 않은 것은?

- ① 세션 41은 LOCK TABLE ... IN SHARE MODE로 테이블 전체에 S(4) 모드 TM 락을 보유(BLOCK=1)한다.
이는 개별 행을 잠그는 TX 락이 아니라 오브젝트(테이블) 수준 락이다.
- ② 세션 55의 UPDATE는 DML이므로 갱신에 앞서 테이블에 RX(3) 모드 TM 락을 요청하는데, 이미 걸린 S(4)와 호환되지 않아 락을 얻지 못하고 REQUEST=3 상태로 대기한다.
- ③ S(4)와 RX(3)는 서로 호환되는 모드이므로, 세션 55의 UPDATE는 대기 없이 TM 락을 획득해 곧바로 갱신을 진행한다.
- ④ 세션 62가 같은 테이블에 다시 LOCK TABLE ... IN SHARE MODE를 실행하면 S(4)끼리는 호환되어 41과 나란히 락을 얻고 대기하지 않지만, 그 상태에서도 세션 55의 RX(3) 요청은 여전히 막힌다.

정답 및 해설

■ 정답 모아보기

1. ①	2. ④	3. ④	4. ①	5. ③
6. ③	7. ④	8. ③	9. ④	10. ②
11. ②	12. ④	13. ②	14. ①	15. ③
16. ①	17. ②	18. ②	19. ①	20. ③

문제 1. 정답 ①

DB_FILE_MULTIBLOCK_READ_COUNT(이하 MBRC)는 한 번의 Multiblock I/O로 읽을 블록 수의 상한일 뿐, 매번 그 수만큼 읽는다는 뜻이 아닙니다. 실제로 한 Call에 묶이는 블록 수는 두 가지 경계에서 그보다 줄어듭니다.

- 경계 1 익스텐트 : Multiblock I/O는 익스텐트 경계를 넘지 못한다.
 익스텐트 끝에 MBRC보다 적은 블록만 남았으면 그만큼만 묶여 읽힌다.
 경계 2 버퍼 캐시: 묶으려는 구간 중간에 이미 캐시에 올라온 블록이 있으면
 그 지점에서 읽기가 끊겨, 앞·뒤로 조각난 더 작은 Call들로 나뉜다.

①은 이 관계를 정반대로 뒤집었습니다. "익스텐트 경계나 캐시 적재 여부와 무관하게 매번 MBRC만큼 읽는다"는 서술은, MBRC를 고정 개수로 오해한 것입니다. MBRC는 한 Call이 넘볼 수 있는 최대치를 정할 뿐이고, 익스텐트 끝과 이미 캐시에 있는 블록이 실제 묶임 크기를 그 아래로 깎습니다. 그래서 같은 Full Scan이라도 캐시 상태와 익스텐트 분할 정도에 따라 Call당 블록 수가 들쭉날쭉합니다.

②·③·④는 옳습니다. Full Scan의 Sequential·Multiblock·**scattered read**(②), 인덱스 경유 Random 액세스의 Single Block·**sequential read**(③), 그리고 MBRC를 키울 때의 I/O Call 절감 경향(④)이 모두 블록 I/O 메커니즘에 부합합니다.

선택지	판정	이유
①	×	MBRC는 한 Call의 블록 수 상한일 뿐입니다. 익스텐트 경계에서 끊기고 캐시에 있는 블록에서 조각나 실제 묶임은 그보다 작아지므로, "경계·캐시와 무관하게 그 수만큼 읽는다"는 방향을 뒤집은 서술입니다
②	○	Full Table Scan은 블록을 물리적 순서로 훑는 Sequential 액세스이고 Multiblock I/O가 동반되며 대기 이벤트는 db file scattered read 입니다
③	○	ROWID로 테이블 블록을 임의로 한 블록씩 집어 오는 Random 액세스는 Single Block I/O이고, 그 이벤트가 이름과 반대인 db file sequential read 입니다
④	○	MBRC를 키우면 한 Call에 묶이는 블록이 늘어 같은 블록 수를 읽을 때 I/O Call 횟수가 줄어드는 경향이 있어 대량 Full Scan에 유리할 수 있습니다

■ 이 문제의 핵심

- MBRC는 한 Multiblock I/O Call의 블록 수 상한이지 고정 개수가 아닙니다. 실제 묶임은 그보다 작아질 수 있습니다.
- Multiblock I/O는 익스텐트 경계를 넘지 못하고, 구간 중간에 이미 캐시에 있는 블록을 만나면 거기서 끊겨 더 작은 Call로 조각납니다.
- Sequential(Full Scan) ↔ Multiblock I/O·**scattered read**, Random(인덱스 경유) ↔ Single Block I/O·**sequential read** — 이벤트 이름이 직관과 반대입니다.
- MBRC를 키우면 Call 횟수가 줄어드는 경향은 있지만, 캐시 상태·익스텐트 분할에 따라 Call당 블록 수는 일정하지 않습니다.

■ 한 줄 정리 **DB_FILE_MULTIBLOCK_READ_COUNT**는 한 Call의 블록 수 상한일 뿐, 익스텐트 경계와 캐시에 있는 블록이 실제 묶임을 그 아래로 깎으므로 "항상 그 수만큼 읽는다"는 서술은 방향을 뒤집은 것입니다.

문제 2. 정답 ④

버퍼 풀 분리의 취지는 성격이 다른 블록끼리 서로 밀어내지 않게 격리하는 것입니다. 세 풀은 별개의 캐시가 아니라 한 버퍼 캐시를 용도별로 구획한 영역이고, 각 구획이 자기 LRU로 독립 관리됩니다.

- DEFAULT : 특별히 지정하지 않은 블록이 캐싱되는 기본 영역
 KEEP : 작고 자주 재사용돼 오래 남기고 싶은 객체용 (충분히 크면 상주에 가까움)
 RECYCLE : 재사용 가능성이 낮은 대용량 객체용 (다른 유용한 블록을 밀어내지 않게 격리)

배정 방법 : 세그먼트의 STORAGE(BUFFER_POOL KEEP | RECYCLE | DEFAULT) 속성으로 지정
관리 : 세 풀 모두 각자의 LRU로 재사용 대상을 고른다 (KEEP도 예외가 아니다)

④는 이 구조를 정확히 짚었습니다. 세 풀은 하나의 캐시를 나눈 영역이고 각자 LRU로 관리되며, 블록이 어느 풀에 갈지는 세그먼트의 **BUFFER_POOL** 속성으로 정합니다. 이름이 아니라 배정 속성이 풀을 가릅니다.

①·②·③은 풀 이름이 불러일으키는 반사적 연상을 그대로 사실로 옳긴 함정입니다. "KEEP=영원히 고정", "RECYCLE=즉시 폐기", "배정은 옵티마이저가 알아서" — 모두 이름·직관에서 곧장 튀어나오지만 실제 동작과 어긋납니다.

선택지	판정	이유
①	×	KEEP 풀도 LRU로 관리됩니다. 이름이 "영구 고정"을 연상시키지만, 공간이 부족하면 KEEP 풀 안에서도 오래된 블록이 교체됩니다. 풀이 충분히 커야 상주에 가까워질 뿐입니다
②	×	RECYCLE 풀도 엄연한 캐시라 올라온 블록을 곧바로 다시 참조하면 재사용할 수 있습니다. 다만 교체가 빨리 일어날 뿐이며, "올리는 즉시 버린다"는 이름에서 온 반사적 오해입니다
③	×	풀 배정은 옵티마이저의 실행 시점 판단이 아니라, 세그먼트의 BUFFER_POOL 저장 속성으로 미리 지정합니다. "옵티마이저가 알아서 배정한다"는 반사적 연상입니다
④	○	세 풀은 한 버퍼 캐시를 용도별로 나눈 영역이며 각자 독립 LRU로 관리되고, 블록이 어느 풀에 캐싱될지는 세그먼트의 BUFFER_POOL 속성으로 지정합니다

▪ 이 문제의 핵심

- KEEP·RECYCLE·DEFAULT는 별개의 캐시가 아니라 한 버퍼 캐시를 용도별로 나눈 영역이며, 각 영역이 독립된 LRU로 관리됩니다.
- KEEP도 LRU 대상입니다. 이름과 달리 영구 고정이 아니라, 풀이 충분히 클 때 오래 남을 뿐입니다.
- RECYCLE도 캐시라 재참조 시 재사용이 가능합니다. 다만 교체가 빨라 오래 머물지 못합니다.
- 블록이 어느 풀에 캐싱될지는 옵티마이저가 아니라 세그먼트의 **BUFFER_POOL** 저장 속성으로 지정합니다.

▪ 한 줄 정리 세 버퍼 풀은 한 캐시를 용도별로 나눈 독립 LRU 영역이고 배정은 세그먼트의 **BUFFER_POOL** 속성으로 정하며, "KEEP=영구 고정·RECYCLE=즉시 폐기·옵티마이저 자동 배정"은 이름에서 온 반사적 오해입니다.

문제 3. 정답 ④

먼저 Call 통계에서 비재귀 CPU의 정체가 어디인지 봅니다.

비재귀 Total = Parse + Execute + Fetch

Count : 8000 + 8000 + 8000 = 24000

CPU : 27.000 + 3.200 + 1.800 = 32.000

비재귀 Parse CPU 비중 = 27.000 / 32.000 × 100 ≈ 84.4% ← 비재귀 CPU의 대부분이 파싱

파스 1회당 CPU = 27.000 / 8,000 ≈ 3.4 ms (소프트파싱은 보통 0.1ms 미만)

비재귀 CPU 32초 중 27초가 Parse입니다. 파스 1회에 3.4ms는 소프트파싱(공유 커서 탐색, 0.1ms 미만) 수준을 크게 넘어 하드파싱의 값입니다. **Misses in library cache during parse: 8000**이 파스 8,000회가 전부 캐시 미스, 즉 하드파싱임을 확정합니다.

하드파싱 비율 = 라이브러리 캐시 미스 ÷ 비재귀 Parse Count

= 8,000 / 8,000 = 100% ← 리터럴 SQL이라 매 문서가 새 커서

다음은 재귀 통계가 무엇인지입니다. 재귀 Call은 애플리케이션 SQL이 아니라, 오라클이 파싱·공간 관리 중에 데이터 디렉터리를 조회하려고 내부적으로 실행한 SQL입니다. 하드파싱 1회마다 obj\\$\col\\$ 같은 디렉터리를 훑으므로, 하드파싱이 많을수록 재귀 호출이 함께 부풀어 오릅니다.

재귀/비재귀 Call 비율 = 재귀 Total Count ÷ 비재귀 Total Count

= 160,000 / 24,000 ≈ 6.67배

재귀 Parse ÷ 비재귀 Parse = 16,000 / 8,000 = 2.0배

재귀 Query 1회당 ≈ 560,000 / 16,000 = 35 블록/파스 ← 파스마다 디렉터리 재조회

핵심은 두 축이 별개라는 점입니다.

축 A - 파싱 부하 : 리터럴 SQL → 하드파싱(Parse CPU 27s, 미스 8,000)

→ 파싱 중 디렉터리 재귀 SQL(재귀 Call 160,000) 파싱

축 B - 데이터 처리량 : 배치가 실제로 다루는 문서 8,000건(비재귀 Execute Rows 8,000)

재귀 통계의 큰 숫자(Rows 520,000·Query 560,000)는 축 A(파싱 부하)의 부산물이지

축 B(애플리케이션 데이터 로직)가 무거워서가 아님.

재귀 Query 560,000·재귀 Rows 520,000이 커 보여도 이것은 디렉터리 조회의 흔적이지, 배치가 처리한 데이터(문서 8,000건)의 양과는 별개입니다. 바인드 변수로 SQL 문자열을 한 종으로 통일하면 하드파싱(축 A)이 사라지고, 그 파생물인 재귀 디렉터리 호출도 함께 붕괴합니다. 따라서 파싱 부하와 데이터 처리량을 구분해 진단한 ④가 옳습니다.

선택지	판정	이유
①	×	재귀 Execute 48,000·재귀 Rows 520,000은 참이지만 그 정체는 상관 서브쿼리 실행이 아니라 하드파싱 8,000회가 파싱 중 훑은 디셔너리 조회입니다. 배치가 실제로 다른 데이터는 비재귀 Execute Rows 8,000(문서 8,000건)뿐이라 데이터 로직은 가볍고, Parse CPU 27초를 '부수 비용'으로 밀어낸 것이 곧 파싱 부하를 데이터 로직으로 오귀속한 별개 축 혼동입니다
②	×	재귀 Disk 920(Fetch 800)·비재귀 Disk 400은 참이나, 920 블록의 물리 읽기로 재귀 Elapsed 16.7초를 만들 수는 없습니다. 재귀 Query 560,000은 파스 16,000회가 파스당 약 35블록씩 디셔너리를 재조회한 흔적이므로, 정체 원인은 버퍼 캐시 미스가 아니라 미스 8,000이 확정하는 하드파싱 자체이고 캐시 확대는 헛짚은 처방입니다
③	×	비재귀 Execute CPU 3.2초·Fetch CPU 1.8초가 작은 것은 데이터 처리가 원래 가벼웠다는 뜻이지 정교한 최적화의 대가가 아닙니다. Misses in library cache during parse: 8000 이 파스 8,000회 전부가 하드파싱(파스당 3.4ms)임을 확정하므로 27초는 정상 비용이 아닌 낭비이고, 재귀 Call도 계획 실행이 아니라 그 하드파싱의 디셔너리 파생 부하라 '이미 최적'은 성립하지 않습니다
④	○	Parse CPU 27초가 비재귀 CPU의 84%(파스당 3.4ms)에 미스 8,000으로 하드파싱이 주범이고, 재귀 Parse 16,000·Call 160,000은 그 하드파싱의 디셔너리 파생 부하입니다. 파싱 부하와 데이터 처리량을 별개 축으로 옮겨 분리해, 바인드 변수가 재귀까지 줄인다고 진단했습니다

■ 이 문제의 핵심

1. 하드파싱 비율 = **Misses in library cache during parse** ÷ 비재귀 Parse Count = 8,000/8,000 = 100%. 파스당 CPU 3.4ms도 소프트파싱(0.1ms 미만)을 크게 넘어 하드파싱을 뒷받침합니다.
2. 재귀(recursive) 통계는 애플리케이션 SQL이 아니라 파싱·디셔너리 내부 SQL입니다. 하드파싱이 많으면 파싱 중 디셔너리 조회로 재귀 호출이 함께 부풀어 오릅니다(재귀/비재귀 Call ≈ 6.67배).
3. 파싱 부하(축 A)와 데이터 처리량(축 B)은 별개입니다. 재귀 Query 560,000·Rows 520,000이 커도 이는 디셔너리 부산물이니, 배치가 다른 문서 8,000건(비재귀 Execute Rows)과 무관합니다.
4. 처방은 서브쿼리 재작성·버퍼 캐시가 아니라 바인드 변수입니다. 하드파싱을 없애면 그 파생물인 재귀 디셔너리 호출도 함께 감소합니다.

■ 한 줄 정리 비재귀 Parse CPU 27초(파스당 3.4ms·미스 8,000)가 하드파싱을 가리키고, 재귀 Call 160,000은 그 하드파싱의 디셔너리 파생 부하이므로, 재귀 통계의 큰 숫자를 데이터 로직 과부하로 오독하지 말고 파싱 부하와 데이터 처리량을 별개 축으로 봐야 합니다.

문제 4. 정답 ①

EXPLAIN PLAN → DBMS_XPLAN.DISPLAY 경로로 보는 계획은 문장을 수행하지 않은 예상 계획이고, 거기 찍히는 행 수는 옵티마이저가 통계로 추정된 카디널리티입니다. 실제로 세어 본 값이 아니므로 통계 상태에 따라 실측과 벌어질 수 있습니다.

DBMS_XPLAN.DISPLAY 가 보여 주는 것

출처 : PLAN_TABLE (EXPLAIN PLAN)이 적재한 예상 계획, 문장 미수행)

행 수 : 통계 기반 추정 카디널리티 (실측 아님)

벌어지는 지정 : 통계 노후 · 컬럼 간 상관관계 · 복합 조건 → 과소/과대 추정

format 인자 (표시 상세도만 조절, 미수행 성격은 그대로)

'BASIC' : 연산·객체명 위주

'TYPICAL' : 기본값 - 카디널리티·Cost 포함

'ALL' : 쿼리 블록/별칭·프로젝션 등 추가

'ADVANCED' : Outline Data까지 ← 그래도 '실측 행 수'는 나오지 않는다

①은 이 성격을 정확히 짚었습니다. **DISPLAY**의 행 수는 추정치이고, 통계가 낡거나 조건 컬럼들이 서로 상관될 때 실제와 크게 어긋날 수 있다는 것이 예상 계획을 해석할 때의 핵심 경계입니다.

②·③·④는 모두 이 경계를 뚫은 서술입니다. **format**은 표시 상세도를 조절할 뿐 미수행 예상 계획을 실측 계획으로 바꾸지 못하고(②), 기본값은 '**BASIC**'이 아니라 '**TYPICAL**'이며(③), Note의 동적 샘플링 표시는 파싱 단계의 추정 보강일 뿐 문장 수행을 뜻하지 않습니다(④).

선택지	판정	이유
①	○	DISPLAY 의 행 수는 통계 기반 추정 카디널리티이며, 통계 노후·컬럼 간 상관관계가 있으면 실제 반환 행 수와 크게 어긋날 수 있습니다. 예상 계획의 성격을 정확히 짚었습니다
②	×	format 은 표시 상세도만 조절합니다. 'ALL'·'ADVANCED'로 올려도 여전히 예상 계획이라 실측 행 수는 나오지 않습니다. 실측을 보려면 실제 수행 후 DISPLAY_CURSOR 가 필요합니다
③	×	format 기본값은 'BASIC'이 아니라 'TYPICAL'이며, 기본값만으로도 카디널리티와 Cost가 출력됩니다. 기본값 경계를 잘못 잡은 서술입니다
④	×	Note의 동적 샘플링 표시는 파싱 단계에서 추정을 보강했다는 뜻일 뿐, 문장을 실제로 수행했다는 의미가 아닙니다. 여전히 예상 계획입니다

▪ 이 문제의 핵심

1. **EXPLAIN PLAN** → **DISPLAY** 계획의 행 수는 통계 기반 추정 카디널리티이며 실측이 아닙니다.
2. 통계 노후·컬럼 간 상관관계가 있으면 추정 행 수가 실제와 크게 벌어질 수 있습니다.
3. **format**은 표시 상세도를 조절할 뿐(**BASIC/TYPICAL/ALL/ADVANCED**), 예상 계획을 실측 계획으로 바꾸지 못합니다. 기본값은 'TYPICAL'입니다.
4. Note의 동적 샘플링 표시는 파싱 시점의 추정 보강이지 문장 수행의 증거가 아닙니다. 실측 행 수는 실제 수행 후 **DISPLAY_CURSOR**로 봐야 합니다.

▪ 한 줄 정리 **DISPLAY**가 보여 주는 행 수는 통계로 추정한 예상 카디널리티라 실제와 어긋날 수 있으며, **format** 상세도나 동적 샘플링 표시가 이 예상 계획을 실측 계획으로 바꾸지는 않습니다.

문제 5. 정답 ③

먼저 BCHR로 논리 대비 물리 읽기의 비율을 확인합니다.

$BCHR = ((Query + Current) - Disk) / (Query + Current) \times 100$
 $= (2,500,000 + 0 - 200,000) / 2,500,000 \times 100$
 $= 2,300,000 / 2,500,000 \times 100 = 92.0\%$
 디스크 물리 읽기 비중 = $200,000 / 2,500,000 \times 100 = 8.0\%$

논리 250만 블록 중 8%를 디스크에서 물리적으로 읽었습니다. 다음은 시간의 정체입니다.

CPU 비중 = $9.000 / 72.000 \times 100 = 12.5\%$ → 나머지 약 87.5%가 대기
 Elapsed - CPU = $72.000 - 9.000 = 63.0$ 초 ← 일한 시간이 아니라 기다린 시간

응답시간의 대부분이 대기이고, 그 대기가 어디서 났는지는 Row Source가 지목합니다.

컨테이너반출입 cr 비중 = $2,360,000 / 2,500,000 \times 100 = 94.4\%$ ← 블록 소비의 대부분
 컨테이너반출입 pr 비중 = $191,000 / 200,000 \times 100 \approx 95.5\%$ ← 물리 읽기의 대부분
 컨테이너반출입 time = 66.5초 (전체 Elapsed 72초의 대부분)
 선택 cr 비중 = $140,000 / 2,500,000 \times 100 \approx 5.6\%$ ← 소량

블록 소비(cr)·물리 읽기(pr)·소요 시간(time)이 세 지표 모두 컨테이너반출입 Full Scan 한 노드에 몰려 있고, HASH JOIN의 **pw=0**이라 해시 영역 디스크 스필도 없습니다. 따라서 병목은 9,000만 건 컨테이너반출입 테이블의 Full Scan I/O이며, ③이 BCHR 해석과 병목 지목을 모두 옳게 했습니다.

선택지	판정	이유
①	×	인용한 time=70.5초·71.9초는 참이지만 그것은 CPU를 쓴 시간이 아니라 자식 노드의 I/O 대기까지 누적된 경과시간입니다. CPU 9초는 Elapsed 72초의 12.5%뿐이라 CPU 바운드가 아니며, 약 87.5%(약 63초)는 컨테이너반출입 Full Scan(pr=191,000)의 I/O 대기이므로 DOP만 올리는 것은 원인 오귀속입니다
②	×	선택이 Full Scan으로 cr=140,000을 쓰는 것은 사실이나 그 비중은 250만 중 5.6%, time도 1.8초뿐입니다. 병목은 cr 94.4%·pr 95.5%·time 66.5초를 차지하는 컨테이너반출입 Full Scan이므로, 선택 인덱스를 만들어도 63초 대기는 겨냥하지 못합니다
③	○	$(2,500,000-200,000)/2,500,000 = 92.0\%$ 로 BCHR이 정확하고, cr(94.4%)·pr(95.5%)·time(66.5초)이 모두 컨테이너반출입 Full Scan에 집중되어 병목 지목이 맞습니다
④	×	HASH JOIN의 pw=0 이 해시 영역 디스크 스필이 없었음을 직접 반증합니다(스필이 있었다면 pw>0). pr 200,000 중 191,000은 컨테이너반출입 Full Scan의 물리 읽기이지 스필 파티션 되읽기가 아니므로, 63초 대기를 PGA 부족으로 오귀속한 헛집은 처방입니다

▪ 이 문제의 핵심

1. BCHR = ((Query+Current)-Disk)/(Query+Current) × 100. 여기서는 (2,500,000-200,000)/2,500,000 = 92.0%입니다.
2. CPU 비중(12.5%)이 낮으면 CPU 바운드가 아닙니다. Elapsed-CPU(63초)가 대기이고, 그 정체는 Row Source의 **pr·time**으로 특정 노드를 지목해야 드러납니다.
3. 병목 노드는 cr·pr·time이 가장 큰 곳 — 여기서는 컨테이너반출입 Full Scan(cr 94.4%, pr 95.5%, time 66.5초)입니다.
4. **pw=0**은 해시 영역 스페일이 없다는 신호 — PGA 부족을 원인으로 오귀속하지 않도록 반드시 확인합니다.
5. 높은 BCHR이 곧 효율은 아닙니다. 92%가 높아 보여도 논리 읽기 250만 블록 자체가 과다하면 캐시를 키워도 낭비는 남습니다.
 - 한 줄 정리 BCHR 92%와 CPU 12.5%까지 옳게 읽어도, cr·pr·time이 모두 컨테이너반출입 Full Scan에 몰리고 pw=0인 사실을 놓치면 병목을 CPU·선박 인덱스·PGA 스페일로 오귀속하게 됩니다.

문제 6. 정답 ③

정비_IX는 (**정비소코드**, **접수일자**, **정비구분**) 순서입니다. 결합 인덱스의 액세스 조건(스캔 시작·종료 지점을 정하는 조건)은 선두 칼럼부터 등치로 이어지다가, 처음 만나는 범위 조건까지입니다.

정비소코드 = 'C07' ← 선두 등치 → 액세스 조건
 접수일자 범위(6월 한 달) ← 첫 범위 칼럼 → 액세스 조건 (여기서 스캔 범위 확정)
 정비구분 = '엔진' ← 범위 칼럼 '뒤' → 인덱스 필터 (범위를 더 좁히지 못함)
 정비상태 = '완료' ← 인덱스에 없음 → 테이블 필터

핵심은 범위 조건 뒤에 오는 칼럼은 액세스 조건이 될 수 없다는 점입니다. 접수일자가 범위(>= ... **AND** < ...)이므로 인덱스 정렬상 정비구분='엔진'인 엔트리는 스캔 구간 안에 흩어져 있어, 정비구분으로는 스캔의 시작·종료 지점(start/stop key)을 앞당기거나 당길 수 없습니다. 그래서 정비구분은 스캔 범위 안에서 각 인덱스 엔트리를 확인해 걸러내는 인덱스 필터로만 작동합니다.

정비상태는 아예 **정비_IX** 구성 칼럼이 아니므로 인덱스에서 평가할 수 없습니다. ROWID로 테이블 블록을 읽어 온 Id 2 TABLE ACCESS BY INDEX ROWID 단계에서 테이블 필터로 걸러집니다.

INDEX RANGE SCAN 정비_IX Card 8,400 ← 액세스(정비소코드=, 접수일자 범위) + 인덱스 필터(정비구분) 통과
 TABLE ACCESS BY INDEX ROWID 정비이력 Card 2,100 ← 테이블 필터(정비상태='완료') 적용 후

③은 액세스 조건·인덱스 필터·테이블 필터 세 층을 정확히 구분합니다. 세 칼럼을 액세스 조건으로 뭉뚱그린 ①, 정비상태를 인덱스 필터로 오해한 ②, 인덱스 반환 건수를 최종 건수로 본 ④는 이 구조와 어긋납니다.

선택지	판정	이유
①	×	정비상태가 Id 2 테이블 필터라는 뒷부분은 맞지만, 정비구분은 범위 칼럼(접수일자) 뒤에 놓여 스캔 시작·종료 지점을 정하지 못하는 인덱스 필터입니다. 세 칼럼을 모두 액세스 조건으로 뭉뚱그린 것이 하위항목 전체화 오류이며, Id 3 Card 8,400을 만든 액세스 조건은 정비소코드·접수일자까지입니다
②	×	정비상태는 정비_IX 구성 칼럼이 아니어서 리프 엔트리에서 평가할 수 없으므로 인덱스 필터가 아니라 Id 2 테이블 액세스 단계의 필터입니다. Id 3의 Card 8,400은 정비상태를 걸러내기 전 값이고, Id 2의 Card 2,100을 만든 것은 접수일자 재확인인 아니라 정비상태='완료' 테이블 필터입니다
③	○	정비소코드(등치)·접수일자(범위)가 액세스 조건, 범위 뒤 정비구분은 인덱스 필터, 인덱스에 없는 정비상태는 테이블 필터 — Card가 8,400→2,100으로 줄어드는 흐름과 일치합니다
④	×	Card 8,400은 정비상태를 아직 반영하지 못한 Id 3 인덱스 단계 값이며, 테이블 필터를 거친 최종 Card는 Id 2의 2,100입니다. Id 1의 2,100은 별도 표기가 아니라 Id 2에서 이미 줄어든 건수가 정비소_PK 1:1 조인으로 그대로 올라온 것이므로, 테이블 액세스 단계의 건수 감소를 부정한 진술이 틀립니다

▪ 이 문제의 핵심

1. 액세스 조건은 선두 칼럼 등치부터 첫 범위 칼럼까지. 정비소코드(=)·접수일자(범위)가 INDEX RANGE SCAN 구간을 확정합니다.
2. 범위 칼럼 뒤 칼럼은 인덱스 필터. 정비구분은 인덱스에 있어도 접수일자 범위 뒤라 스캔 범위를 좁히지 못하고, 구간 안에서 엔트리마다 확인만 됩니다.
3. 인덱스에 없는 칼럼은 테이블 필터. 정비상태는 ROWID로 테이블을 읽은 뒤 Id 2에서 걸러집니다.
4. 세 층의 건수 감소 지점이 다릅니다. 인덱스 단계 8,400건과 테이블 단계 후 2,100건의 차이가 곧 테이블 필터의 효과입니다.

- 한 줄 정리 선두 등치(정비스코드)와 첫 범위(접수일자)까지가 액세스 조건이고, 범위 뒤 정비구분은 인덱스 필터, 인덱스에 없는 정비상태는 테이블 필터라, Card가 8,400에서 2,100으로 줄어듭니다.

문제 7. 정답 ④

트레이스의 세 숫자를 나란히 놓습니다.

600,000 ← 인덱스에서 얻은 ROWID 수 (INDEX RANGE SCAN 의 Rows)
 271,800 ← 테이블 액세스 누적 블록 I/O (TABLE ACCESS 의 cr)
 1,800 ← 인덱스 스캔에 든 블록 (INDEX RANGE SCAN 의 cr)

부모(TABLE ACCESS) cr에는 자식(INDEX) cr이 누적돼 있으므로, 순수 테이블 랜덤 블록을 분리합니다.

테이블 랜덤 블록 = 271,800 - 1,800 = 270,000
 랜덤 블록 비중 = 270,000 / 271,800 ≈ 99.3% ← cr의 거의 전부가 테이블 랜덤

클러스터링 팩터(CF)의 밀도를 봅니다. 테이블의 행 밀도는 10,000,000 / 100,000 = 100행/블록입니다.

CF 밀도 ≈ 테이블 랜덤 블록 ÷ ROWID 수 = 270,000 / 600,000 = 0.45 (ROWID당 블록)
 CF 추정 ≈ 0.45 × 10,000,000 = 4,500,000
 → 테이블 블록 100,000의 약 45배 (100행/블록 × 0.45)
 → 나쁜지만 최악(1.0)은 아닌 중간값

읽은 행 비율은 600,000 / 10,000,000 = 6.0%로 흔히 '인덱스가 유리'로 보이는 범위지만, 나쁜 CF 탓에 단일블록 랜덤 읽기가 27만 블록에 이릅니다. 이제 세 경로를 손익분기점으로 비교합니다.

현재 경로(인덱스+테이블) = 270,000(랜덤) + 1,800(인덱스) = 271,800 블록
 Full Scan = 100,000 블록 (multiblock)
 커버링 인덱스 스캔 ≈ 인덱스만 읽음 → 컬럼 2개 추가로 리프가 넓어져도 수천 블록대

Elapsed 38.0초 대비 CPU 5.4초의 약 32.6초 간극은 물리 읽기(Disk 60,000) 대기이며, 그 뿌리는 테이블 랜덤 액세스입니다. SELECT가 **점검일자·점검결과** 두 컬럼뿐이므로 이 둘을 인덱스에 엮은 커버링 인덱스면 **TABLE ACCESS BY INDEX ROWID**가 사라져 랜덤 블록 27만이 통째로 없어집니다. cr은 인덱스만 읽는 수준으로 떨어져 Full Scan 100,000보다도 적습니다. 병목은 인덱스가 아니라 테이블 랜덤 액세스이고, 최적 처방은 테이블 액세스 제거(커버링)이므로 ④가 옳습니다.

선택지	판정	이유
①	×	(271,800-60,000)/271,800 ≈ 77.9%도, 38.0-5.395 ≈ 32.6초가 대기라는 것도 맞습니다. 그러나 낭비의 정체는 캐시 크기가 아니라 논리 읽기 271,800 자체(나쁜 CF)입니다. 캐시를 키워 pr 60,000을 0으로 만들어도 27만 블록의 랜덤 액세스와 그 CPU·래치 비용은 그대로 남으므로 근본 원인 오귀속입니다
②	×	271,800 > 100,000으로 손익분기점을 넘었다는 앞부분은 맞지만, 뒷부분의 단정이 틀립니다. SELECT가 점검일자·점검결과 두 컬럼뿐이라 커버링 인덱스가 TABLE ACCESS BY INDEX ROWID를 통째로 없애 cr을 현재 인덱스 cr 1,800 수준에서 리프가 넓어진 만큼만 늘어난 수천 블록대로 낮추므로, Full Scan 100,000보다 오히려 유리한 선택지가 남아 있습니다
③	×	270,000/600,000 = 0.45라는 계산 자체는 맞으나 해석이 반대입니다. 100행/블록인 테이블에서 ROWID당 0.45블록은 CF가 테이블 블록의 약 45배라는 뜻으로 나쁜 쪽입니다. 인덱스 노드는 cr=1,800·time=0.62초뿐이라 38초를 만들 수 없고, 대기의 정체는 테이블 랜덤 액세스의 물리 I/O입니다
④	○	테이블 랜덤 블록 270,000 / ROWID 600,000 = 0.45로 CF는 테이블 블록의 약 45배(나쁨)이고 cr의 99.3%가 랜덤입니다. 두 컬럼만 읽으므로 커버링으로 테이블 액세스를 없애면 cr이 현재 인덱스 cr 1,800 수준에서 리프가 넓어진 만큼 늘어도 수천 블록대에 그쳐 Full Scan 100,000보다 유리합니다

■ 이 문제의 핵심

- 테이블 랜덤 블록 = TABLE ACCESS cr - INDEX cr = 271,800 - 1,800 = 270,000. 부모 cr에는 자식 인덱스 cr이 누적돼 있으니 반드시 빼서 분리합니다.
- CF 밀도 ≈ 테이블 랜덤 블록 ÷ ROWID 수. 0.45는 100행/블록 기준 테이블 블록의 약 45배로, 최악(1.0)은 아니어도 분명히 나쁜 값입니다.
- cr의 99.3%가 테이블 랜덤이므로 병목은 인덱스가 아니라 테이블 랜덤 액세스입니다.
- 손익분기점 비교: 현재 271,800 > Full Scan 100,000 > 커버링 수천 블록대(현재 인덱스 cr 1,800에서 리프가 넓어진 만큼 증가). 읽는 컬럼이 적으면 Full Scan보다 커버링이 더 유리합니다.
- 높은 논리 읽기 자체가 낭비라면 캐시 확대로는 해결되지 않습니다 — 테이블 액세스를 제거(커버링)해야 랜덤 블록 27만이 사라집니다.

- 한 줄 정리 ROWID 60만에 테이블 랜덤 27만이면 CF는 테이블 블록의 약 45배로 나쁘고, SELECT가 두 컬럼뿐이라 커버링 인덱스로 테이블 액세스를 없애는 것이 Full Scan보다도 유리한 처방입니다.

문제 8. 정답 ③

두 인덱스는 리프 레벨에 무엇이 있는가로 같습니다. 이 차이가 테이블을 되짚는 추가 액세스의 유무를 결정합니다.

일반 B-Tree 인덱스 (오라클 힙 테이블 인덱스 · SQL Server 비클러스터형)

리프 : 인덱스 키 + 행 위치(ROWID/RID)

조회 : 인덱스에 없는 컬럼이 필요하면 ROWID/RID로 테이블을 되짚는

추가 Random 액세스(테이블 액세스 / key lookup)가 발생

클러스터형 인덱스(SQL Server) · IOT(오라클)

리프 : 인덱스 키 순서로 정렬된 '행 데이터 그 자체'

조회 : 키로 리프에 도달하면 거기 행이 있으므로 되짚기 불필요

제약 : 물리적 정렬은 하나뿐 → 테이블당 하나만 가능

③은 이 성질을 정확히 짚었습니다. IOT·클러스터형 인덱스는 리프가 곧 데이터라, 키로 찾아 리프에 닿으면 그 자리에서 행을 얻어 별도의 ROWID/RID 되짚기가 필요 없습니다.

①·②·④는 각 개념의 경계를 뭉개 함정입니다.

①은 클러스터형의 성질(리프=데이터)을 비클러스터형에 갖다 붙였습니다. 비클러스터형 리프에는 키와 RID만 있어, 인덱스에 없는 컬럼이 필요하면 테이블을 되짚어야 합니다.

②는 "리프가 곧 데이터"라는 성질에서 물리 정렬이 하나뿐이라는 제약을 빠뜨렸습니다. 클러스터형 인덱스는 테이블당 하나이므로 "여러 개를 만들어 조회마다 다른 정렬본을 고른다"는 성립하지 않습니다.

④는 Index Unique Scan과 Range Scan의 경계를 지웠습니다. Unique Scan은 유일성이 보장돼 최대 한 건을 찾으면 즉시 멈춥니다. 여러 건을 이어 훑는 것은 Range Scan입니다.

선택지	판정	이유
①	×	리프에 행 데이터를 담는 것은 클러스터형입니다. 비클러스터형 리프에는 키+RID만 있어, 인덱스에 없는 컬럼이 필요하면 테이블을 되짚어야 합니다. 두 인덱스의 경계를 뒤섞었습니다
②	×	클러스터형 인덱스는 물리 정렬이 하나뿐이라 테이블당 하나만 만들 수 있습니다. "여러 개를 만들어 조회마다 다른 정렬본을 고른다"는 성립하지 않습니다
③	○	IOT·클러스터형 인덱스는 리프에 행 데이터를 키 순서로 담으므로, 키로 리프에 도달하면 그 자리에서 행을 얻어 ROWID/RID로 힙 테이블을 되짚을 필요가 없습니다
④	×	Index Unique Scan은 유일성이 보장돼 최대 한 건을 찾으면 즉시 종료합니다. 여러 건을 리프에서 이어 읽는 것은 Range Scan이며, Unique Scan과 경계가 다릅니다

■ 이 문제의 핵심

1. 인덱스는 리프에 무엇이 있는가로 같습니다. 일반 B-Tree·비클러스터형은 키+ROWID/RID, 클러스터형·IOT는 행 데이터 그 자체입니다.
2. 클러스터형·IOT는 키로 리프에 닿으면 되짚기가 불필요합니다. 비클러스터형은 인덱스에 없는 컬럼이 필요하면 테이블을 되짚습니다.
3. 클러스터형 인덱스는 물리 정렬이 하나뿐이라 테이블당 하나만 만들 수 있습니다.
4. Index Unique Scan = 최대 한 건 찾고 즉시 종료. 여러 건을 이어 훑는 것은 Range Scan입니다.

■ 한 줄 정리 IOT·클러스터형 인덱스는 리프가 곧 데이터라 키 조회 시 힙 테이블 되짚기가 필요 없으며, 비클러스터형은 키+RID만 담고 클러스터형은 테이블당 하나, Unique Scan은 한 건에서 멈춘다는 경계를 지켜야 합니다.

문제 9. 정답 ④

결합 인덱스에서 어떤 컬럼이 스캔 범위를 좁히는 액세스 조건으로 작동하려면, 인덱스 안에서 그 컬럼 앞에 놓인 컬럼들이 모두 등가(=) 조건으로 묶여 있어야 합니다. 앞 컬럼 중 하나라도 범위 조건이면 그 뒤 컬럼은 정렬이 흐트러져 시작·종료 지점을 좁히지 못하고, 리프를 훑으며 걸러 내는 필터 조건으로만 쓰입니다.

인덱스 (A, B, C) 기준

where A = :a and B = :b and C = :c → A·B·C 모두 액세스 조건, 시작·끝 좁게 확정

where A = :a and B > :b and C = :c → C는 앞의 B가 범위라 액세스 조건이 못 됨
C는 리프를 훑으며 거르는 '필터'로만 작동

where B = :b (A 조건 없음) → 선행 A가 빠져 B는 시작점을 못 잡음

④는 이 부분적 성립 조건을 전제로 확대한 함정입니다. "컬럼이 조건절에 등장하기만 하면 위치·앞 조건과 무관하게 액세스 범위를 좁힌다"는 서술은, 실제로는 앞선 컬럼들이 모두 등가일 때만 성립하는 성질을 모든 경우로 늘려 붙인 것입니다. 위치가

뒤이거나 앞에 범위 조건이 끼면 그 컬럼은 범위를 좁히지 못하고 필터로만 작동합니다.

①·②·③은 옳습니다. 선행 컬럼에 등가·고선택도 컬럼을 두는 기준(①), 컬럼 추가에 따르는 인덱스 크기·DML 부하 증가(②), 조희 패턴에 맞춘 컬럼 순서 설계(③)가 모두 결합 인덱스 설계의 정석을 짚습니다.

선택지	판정	이유
①	○	선행 컬럼에 등가 조건으로 자주 쓰이고 변별력이 높은 컬럼을 두면 스캔 시작·종료 지점을 좁게 잡아 훑는 리프 블록 수를 줄일 수 있습니다
②	○	컬럼을 더하면 리프 엔트리가 길어져 인덱스가 커지고, 그 테이블의 DML마다 갱신·정렬할 인덱스 데이터가 늘어 DML 부하가 증가합니다
③	○	같은 컬럼 집합이라도 조희 패턴에 따라 순서를 달리 설계해야 하며, 등가 조건으로 자주 쓰이는 컬럼을 선행에 두는 편이 스캔 범위를 좁히는 데 유리합니다
④	×	추가한 컬럼이 액세스 범위를 좁히는 것은 앞선 컬럼이 모두 등가 조건일 때뿐입니다. 위치·앞 조건과 무관하게 범위를 좁혀 준다는 것은 부분 성립 조건을 전체로 확대한 서술입니다

■ 이 문제의 핵심

1. 결합 인덱스에서 컬럼이 액세스 조건으로 작동하려면 그 앞의 컬럼들이 모두 등가(=) 조건이어야 합니다.
2. 앞 컬럼 중 하나라도 범위 조건이면 그 뒤 컬럼은 스캔 범위를 못 좁히고 필터 조건으로만 쓰입니다.
3. 그래서 선행 컬럼에는 등가 조건으로 자주 쓰이고 선택도 높은 컬럼을 두고, 조희 패턴에 맞춰 순서를 설계합니다.
4. 컬럼 추가는 공짜가 아닙니다 — 인덱스가 커지고 DML 부하가 조희 종류와 무관하게 늘어납니다.

■ 한 줄 정리 추가한 컬럼이 스캔 범위를 좁히는 것은 앞선 컬럼이 모두 등가 조건일 때뿐이며, 위치·앞 조건과 무관하게 좁혀 준다는 서술은 부분 성립 조건을 전체로 확대한 것입니다.

문제 10. 정답 ②

NESTED LOOPS(Id 1) 아래 두 자식의 위치와 접근 경로를 읽습니다.

```

Id 1  NESTED LOOPS
Id 2  TABLE ACCESS BY INDEX ROWID 수강신청  ← 먼저 오는 자식 = 드라이빙(선행)
Id 3  INDEX RANGE SCAN 수강_IX              ← 드라이빙 집합 축소 (530건)
Id 4  TABLE ACCESS BY INDEX ROWID 강좌      ← 나중 자식 = 인너(후행), 반복 수행
Id 5  INDEX UNIQUE SCAN 강좌_PK              ← 인너 접근 경로 (드라이빙 건마다 재실행)

```

NESTED LOOPS에서 먼저 나열된 자식이 드라이빙(선행) 집합입니다. 여기서는 수강신청이 **수강_IX**(Id 3)로 **개설학기='2026-2' AND 학과코드='CSE'** 약 530건까지 좁혀지고, 그 각 행마다 인너 강좌를 **강좌_PK** INDEX UNIQUE SCAN(Id 5)으로 반복 탐색합니다. 즉 인너 접근 횟수는 드라이빙 수강신청 건수(약 530)만큼이며, 강좌가 12만 건이든 120만 건이든 반복 횟수 자체는 드라이빙이 결정합니다.

인너 반복 횟수 = 드라이빙(수강신청) 건수 = 약 530회
 ≠ 인너(강좌) 테이블 크기 12만

또한 NL은 드라이빙 앞쪽 행부터 하나씩 조인해 결과를 곧바로 흘려보내므로, 전체를 다 완성하지 않고도 첫 행을 빠르게 낼 수 있습니다. 그래서 상위 일부(예: 화면 첫 페이지)만 필요할 때 전체 결과를 만들지 않고 멈추는 부분범위처리에 유리합니다.

②가 드라이빙 지목·반복 구조·부분범위처리 이점을 모두 정확히 서술합니다.

선택지	판정	이유
①	×	인너 12만 건, 드라이빙 530건이라는 수치는 맞지만 병목을 영뚱한 요인에 결부했습니다. Id 5 강좌_PK UNIQUE SCAN의 반복 횟수는 인너 크기가 아니라 드라이빙 530건이 결정하고 UNIQUE SCAN 1회 비용은 인덱스 높이만큼으로 고정이므로, 강좌를 줄여도 반복 부담은 그대로입니다
②	○	먼저 나온 자식 수강신청(Id 2)이 드라이빙이고, 530건 각각에 강좌_PK UNIQUE SCAN(Id 5)이 반복됩니다. NL은 앞쪽부터 조인해 부분범위처리에 유리합니다
③	×	인너를 한 번에 읽어 정렬·적재한 뒤 병합하는 것은 소트 머지 조인의 서술입니다. Id 1이 NESTED LOOPS이고 플랜에 SORT JOIN도 HASH JOIN 노드도 없으므로, Id 5 강좌_PK UNIQUE SCAN은 한 차례가 아니라 드라이빙 530건마다 반복 수행됩니다
④	×	드라이빙과 인너를 뒤바꿨습니다. NESTED LOOPS에서 먼저 나열된 자식 Id 2가 수강신청이므로 드라이빙은 수강신청이고, 530건은 반복 탐색의 누적이 아니라 수강_IX 가 Id 3에서 한 번에 좁힌 드라이빙 집합입니다. 부분범위처리 이득도 인너 건수가 아니라 상위 일부만 필요할 때 커집니다

■ 이 문제의 핵심

1. NESTED LOOPS의 먼저 나열된 자식이 드라이빙(선행) 집합입니다. 여기서는 수강신청(id 2)이 드라이빙입니다.
2. 인너 접근은 드라이빙 건수만큼 반복됩니다. **강좌_PK UNIQUE SCAN**(id 5)이 수강신청 530건마다 재실행됩니다.
3. 반복 횟수는 인너 크기가 아니라 드라이빙 건수가 결정합니다. 강좌가 12만 건이어도 조인 반복은 530회입니다 — 병목을 인너 크기로 오귀속하지 않도록 주의합니다.
4. NL은 부분범위처리에 유리합니다. 드라이빙 앞쪽부터 조인해 상위 일부만 필요할 때 첫 행을 빠르게 냅니다.
 - 한 줄 정리 NESTED LOOPS에서 위쪽 자식 수강신청이 드라이빙이고 530건마다 **강좌_PK UNIQUE SCAN**이 반복되며, 반복 횟수는 인너 강좌 크기가 아니라 드라이빙 수강신청 건수가 결정하고 NL은 부분범위처리에 유리합니다.

문제 11. 정답 ②

Top-N 쿼리의 성능은 상위 N건을 얻으려고 전체를 정렬하느냐, 아니면 앞쪽 N건만 읽고 멈추느냐로 갈립니다.

핵심은 정렬 순서를 인덱스가 대신 제공하는가입니다. **ORDER BY** 컬럼과 인덱스 정렬 순서가 일치하면, 옵티마이저는 정렬 오퍼레이션을 두지 않고 인덱스를 앞에서부터 훑기만 하다가 **ROWNUM**이 N에 도달하는 순간 스캔을 중단(COUNT STOPKEY)합니다. 뒤쪽 방대한 데이터는 읽지 않으므로, 데이터가 아무리 많아도 읽는 양은 앞쪽 N건 근처로 고정됩니다. 이것이 부분범위 처리이며 ②가 이 원리를 정확히 진술합니다.

나머지는 부분범위 중단이 작동하는 경계를 뭉개 진술입니다.

- ①: 인덱스가 정렬 순서를 제공하면 정렬 단계 자체가 사라집니다(STOPKEY). "언제나 전체를 정렬한 뒤 자른다"는 정렬을 인덱스로 회피하는 경우를 지워 버린 오류입니다.
- ③: **ORDER BY**가 없으면 옵티마이저는 어떤 순서로도 정렬해 줄 의무가 없어, 인덱스가 있더라도 "먼저 만난 N건"이 담길 뿐 값 순 상위 N건이 보장되지 않습니다. Top-N을 정확히 얻으려면 반드시 **ORDER BY**가 함께 있어야 합니다.
- ④: **ROWNUM STOPKEY**는 상위 로우소스 전체에 걸리므로, 드라이빙을 인덱스 순으로 읽는 NL 조인이라면 N건을 채우는 순간 조인 자체가 멈춥니다. inner 반복도 그 지점에서 끊겨 오히려 반복 횟수가 크게 줄어듭니다.

선택지	판정	이유
①	×	정렬 순서를 주는 인덱스가 있으면 STOPKEY로 정렬 단계가 사라집니다. "언제나 전체 정렬 후 자른다"는 인덱스 정렬 회피 경우를 지운 오류입니다
②	○	정렬 순서를 주는 인덱스가 있으면 ROWNUM <= N 은 앞에서 N건을 채운 뒤 스캔을 멈춰, 정렬 없이 상위 N건만 읽고 끝납니다
③	×	ORDER BY 가 없으면 인덱스가 있어도 값 순 상위 N건이 보장되지 않고, 먼저 조회된 임의의 N건이 담깁니다
④	×	ROWNUM STOPKEY 는 조인 로우소스에 걸려 N건을 채우면 조인이 멈추므로, NL의 inner 반복 횟수도 그 지점에서 줄어듭니다

▪ 이 문제의 핵심

1. 인덱스가 정렬 순서를 주면 Top-N은 정렬 없이 앞 N건만 읽고 멈춘다(COUNT STOPKEY). 이것이 부분범위 처리입니다.
2. 읽는 양이 N건 근처로 고정되므로, 뒤쪽 데이터가 아무리 많아도 비용이 커지지 않습니다.
3. **ORDER BY**가 없으면 Top-N은 성립하지 않는다. 인덱스가 있어도 값 순 상위 N건은 보장되지 않습니다.
4. NL 조인에 STOPKEY가 걸리면 N건 시점에 조인이 멈춰 inner 반복 횟수까지 줄어듭니다.

▪ 한 줄 정리 정렬 순서를 제공하는 인덱스가 있으면 **ROWNUM <= N**은 앞에서 N건만 읽고 멈추는 부분범위 처리로 동작하며, "전체 정렬 후 절단"·"**ORDER BY** 없이도 Top-N 보장"·"NL 조인은 중단 불가"는 부분범위 중단의 작동 경계를 뭉개 오해입니다.

문제 12. 정답 ④

RBO와 CBO는 계획을 고르는 축 자체가 다른 독립된 방식입니다.

RBO : 규칙 축 - 미리 정해진 액세스 경로 우선순위로 결정, 통계 무시

CBO : 비용 축 - 통계로 각 계획의 예상 비용을 산정해 최소 비용을 선택

④는 이 서로 독립인 두 축을 인과로 엮은 별개 축 혼동입니다. CBO는 RBO의 규칙 우선순위표를 먼저 적용하지 않습니다. CBO는 후보 계획들의 비용을 통계로 매겨 공장 최소 비용을 고를 뿐이며, "규칙으로 순위를 정한 뒤 통계로 재확인"하는 단계는 존재하지 않습니다. 두 방식은 규칙 축을 공유하지 않으므로 ④가 틀린 진술입니다.

나머지는 모두 옳습니다.

- ①: RBO의 정의 그대로입니다. 통계를 보지 않고 정해진 규칙 순위로만 경로를 정합니다.

- ②: RBO 규칙 순위에서는 유일 인덱스 등치 액세스가 전체 스캔보다 우선하므로, 통계상 전체 스캔이 유리한 경우에도 규칙이 인덱스를 택합니다. RBO가 통계를 무시하기 때문에 생기는 대표적 한계입니다.
- ③: CBO는 통계 기반이라 통계가 실제와 어긋나면 비용 추정이 빗나가 나쁜 계획을 고를 수 있습니다.

선택지	판정	이유
①	○	RBO는 정해진 액세스 경로 우선순위 규칙으로 계획을 정하고 통계는 쓰지 않습니다
②	○	RBO 순위에서 유일 인덱스 등치 액세스가 전체 스캔보다 우선하므로, 전체 스캔이 유리해도 규칙상 인덱스가 선택됩니다
③	○	CBO는 통계로 비용을 산정해 최소 비용 계획을 고르므로, 통계가 부정확하면 계획 품질이 나빠집니다
④	×	CBO는 RBO의 규칙 우선순위를 거치지 않고 비용으로 곧장 계획을 고릅니다. 규칙 축과 비용 축은 독립이며 공유되지 않습니다

■ 이 문제의 핵심

1. RBO = 규칙 축, CBO = 비용 축. 계획을 고르는 근거가 서로 독립입니다.
 2. RBO는 통계를 무시하므로, 전체 스캔이 유리한 상황에서도 규칙 순위대로 인덱스를 택하는 한계가 있습니다.
 3. CBO는 통계로 비용을 산정해 최소 비용을 고르므로, 통계 정확도가 계획 품질을 좌우합니다.
 4. CBO는 RBO의 규칙표를 거치지 않는다. "규칙으로 순위 매긴 뒤 비용으로 재확인"은 독립인 두 축을 억지로 엮은 오류입니다.
- 한 줄 정리 RBO는 통계를 무시하고 정해진 규칙 순위로, CBO는 통계 기반 비용으로 계획을 고르는 독립된 두 방식이며, CBO가 RBO의 규칙 우선순위를 먼저 적용한다는 진술은 별개의 두 축을 인과로 엮은 오류입니다.

문제 13. 정답 ②

v\$sql 요약이 곧 근거입니다. 리터럴만 다른 세 SQL이 **CURSOR_SHARING=FORCE** 아래에서 SQL_ID 하나(h7k3...)로 합쳐졌고, SQL_TEXT의 리터럴 자리가 **:"SYS_B_0"**로 바뀌었습니다. **PARSE_CALLS 3 / EXECUTIONS 3 / LOADS 1**은 세 번 파싱을 요청했지만 커서는 한 번만 적재(하드파싱 1회) 하고 이후엔 공유(소프트파싱)했다는 뜻입니다.

'S1' ↙
 'S2' +→ FORCE가 리터럴을 **:"SYS_B_0"**로 치환 → 텍스트 동일 → SQL_ID 하나
 'S3' ↘ LOADS 1 (하드파싱 1회), EXECUTIONS 3 (세 번 실행 누적)

CURSOR_SHARING=FORCE가 하는 일은 리터럴 상수를 바인드 변수로 치환하는 것뿐입니다. 키워드 대소문자·공백·힌트는 건드리지 않으므로, 리터럴 외의 요소가 다르면 텍스트는 여전히 다른 것으로 간주되어 별도 커서로 갈립니다. ②는 "FORCE가 대소문자·공백·힌트까지 정규화해 합쳐 준다"는 정규화 가정의 오류입니다. 만약 ②처럼 정규화까지 한다면 그것은 FORCE의 동작 범위를 넘는 진술입니다. 그래서 부적절한 것은 ②입니다.

①④는 요약표가 보여 주는 리터럴 치환·커서 공유·EXECUTIONS 누적을 정확히 서술하고, ③은 공유의 대가로 따라오는 바인드 변수 엮보기 부작용을 옳게 짚습니다.

선택지	판정	이유
①	○	FORCE가 리터럴을 :"SYS_B_0" 로 치환해 값만 다른 세 SQL이 h7k3... 커서 하나로 공유됩니다. 요약표의 SQL_ID 단일화가 그 증거입니다
②	×	FORCE는 리터럴만 바인드로 치환할 뿐 대소문자·공백·힌트는 정규화하지 않습니다. 그 요소가 다르면 텍스트가 달라 여전히 별도 커서입니다
③	○	리터럴을 바인드화하면 하드파싱은 줄지만, 편중된 칼럼에서는 첫 바인드 값에 맞춘 계획을 이후 값이 재사용하는 엮보기 부작용이 생길 수 있습니다
④	○	EXACT였다면 세 리터럴이 각각 다른 SQL_ID로 하드파싱되지만, FORCE에선 :"SYS_B_0" 커서 하나만 남고 EXECUTIONS가 3으로 누적됩니다 — 요약표 그대로입니다

■ 이 문제의 핵심

1. **CURSOR_SHARING=FORCE**는 리터럴 상수를 시스템 생성 바인드 변수로 치환합니다. 값만 다른 SQL을 하나의 커서로 공유시킵니다.
2. FORCE는 대소문자·공백·힌트를 정규화하지 않습니다. 리터럴 외의 요소가 다르면 텍스트가 달라 여전히 별도 SQL_ID입니다.
3. LOADS는 하드파싱 횟수, EXECUTIONS는 실행 횟수입니다. LOADS 1·EXECUTIONS 3은 한 번 적재해 세 번 공유했다는 뜻입니다.

4. 공유에는 대가가 따릅니다 — 값이 편중된 칼럼에서 바인드로 묶으면 첫 실행의 계획이 굳어지는 바인드 변수 엮이기 부작용을 감수해야 합니다.

- 한 줄 정리 **CURSOR_SHARING=FORCE**는 리터럴만 바인드 변수로 치환해 값만 다른 SQL을 한 커서로 공유시킬 뿐, 대소문자·공백·힌트까지 정규화하지는 않습니다 — "그런 요소까지 정규화해 합쳐 준다"는 ②가 부적절합니다.

문제 14. 정답 ①

상관 서브쿼리를 그대로 두면 옵티마이저는 바깥 행마다 서브쿼리를 반복 수행하는 필터 방식에 갇힙니다. 조인 순서도 조인 방식도 고를 여지가 없습니다.

서브쿼리 Unnesting은 이 중첩 구조를 풀어 서브쿼리를 조인으로 승격시키는 변환입니다. **EXISTS** 상관 서브쿼리는 세미(semi) 조인으로 바뀌어, 바깥 집합의 각 행에 대해 짝이 하나라도 있으면 통과시키고 중복 증식은 막는 형태가 됩니다. 조인으로 바뀌고 나면 옵티마이저는 어느 쪽을 먼저 읽을지(순서)와 NL·해시·소트머지 중 무엇으로 이을지(방식)를 비용으로 선택할 수 있습니다. ①이 이 원리를 정확히 진술합니다.

나머지는 키워드에서 오는 반사적 연상을 노린 함정입니다.

②: "부정 = 조인 승격 불가"로 연상하기 쉽지만, **NOT EXISTS**는 안티(anti) 조인으로 Unnesting됩니다. 짝이 없음을 확인하는 판정은 안티 조인이 그대로 수행하므로 필터 형태에 갇히지 않으며, 조인으로 바뀐 뒤에는 **EXISTS**와 똑같이 조인 방식·순서 선택이 열립니다.

③: 둘 다 "조건·서브쿼리를 다룬다"는 인상으로 한 절차의 앞뒤처럼 묶었지만, 조건절 이행은 등치 연쇄에서 새 조건을 유도하는 변환($A=B \wedge B=10 \rightarrow A=10$)이고, Unnesting은 중첩 서브쿼리를 조인으로 바꾸는 변환입니다. 서로 독립이라 서브쿼리가 없는 단순 조인 SQL에서도 조건절 이행은 그대로 일어납니다.

④: "서브쿼리 = Unnesting" 반사 연결이지만, 세미/안티 조인으로의 Unnesting 대상은 **WHERE** 절의 **EXISTS·IN** 같은 상관 서브쿼리입니다. SELECT 절의 스칼라 서브쿼리는 행마다 값 하나를 돌려주는 별개 형태여서 세미 조인의 통과·차단 의미와 맞지 않고, **EXISTS·IN**을 그 변형으로 두는 것은 대표 사례를 거꾸로 세운 서술입니다.

선택지	판정	이유
①	○	EXISTS 상관 서브쿼리를 세미 조인으로 Unnesting하면 필터 반복 대신 조인 방식·순서를 비용으로 고를 수 있습니다
②	×	NOT EXISTS 는 안티(anti) 조인으로 Unnesting됩니다. '짝이 하나도 없음'이라는 판정 자체를 안티 조인이 수행하므로 조인 승격이 막히지 않고, 필터 형태에만 갇힌다는 진술도 조인 방식·순서를 못 고른다는 결론도 성립하지 않습니다
③	×	조건절 이행은 등치 연쇄에서 새 조건을 유도하는 변환($A=B \wedge B=10 \rightarrow A=10$)이고 Unnesting은 중첩 서브쿼리를 조인으로 바꾸는 변환입니다. 서로 독립이라 앞뒤 단계가 아니며, 서브쿼리가 없는 SQL에서도 조건절 이행은 일어납니다
④	×	세미/안티 조인 Unnesting의 대상은 WHERE 의 EXISTS·IN 상관 서브쿼리입니다. SELECT 절 스칼라 서브쿼리는 행마다 값 하나를 돌려주는 별개 형태라 세미 조인의 통과·차단 의미와 맞지 않고, EXISTS·IN 을 그 변형으로 둔 것은 대표 사례를 거꾸로 세운 서술입니다

▪ 이 문제의 핵심

1. Unnesting은 서브쿼리를 조인으로 승격시켜, 필터 반복 대신 조인 방식·순서를 비용으로 고를 수 있게 합니다.
2. **EXISTS** → 세미 조인, **NOT EXISTS** → 안티 조인. 부정 조건도 조인으로 풀립니다.
3. 조건절 이행 ≠ Unnesting. 이행은 새 조건을 유도하고, Unnesting은 중첩 구조를 조인으로 바꾸는 독립 변환입니다.
4. 세미/안티 Unnesting 대상은 **WHERE**의 상관 서브쿼리이지, SELECT 절 스칼라 서브쿼리가 아닙니다.

▪ 한 줄 정리 **EXISTS/NOT EXISTS** 상관 서브쿼리는 세미·안티 조인으로 Unnesting되어 조인 방식·순서 선택을 여는 변환이며, 이를 조건절 이행과 한 기법으로 묶거나 스칼라 서브쿼리·부정 조건에 반사적으로 갖다 붙이는 것은 키워드에 이끌린 연상 오류입니다.

문제 15. 정답 ③

수정 가능 조인 뷰의 핵심 규칙은 key-preserved 테이블의 칼럼만 수정할 수 있다는 것입니다. key-preserved란 조인 결과에서도 그 테이블의 각 행이 최대 한 번만 나타나 원래의 유일성이 보존되는 쪽을 말합니다.

- (가) 상여 b 사원 s ON b.사원번호 = s.사원번호
 사원.사원번호 = PK(유일) → 상여 한 행에 사원 한 행만 매칭
 ⇒ 상여 = key-preserved(수정 가능), 사원 = 비보존(수정 불가)
- (나) MERGE : WHEN MATCHED / WHEN NOT MATCHED 는 각각 선택적

둘 중 하나만 써도 유효 (UPDATE만, 또는 INSERT만 하는 MERGE도 가능)

③은 두 군데를 뒤집었습니다. 첫째, 조인 뷰에서 수정할 수 있는 것은 key-preserved 테이블(상여)의 칼럼뿐이며, 비보존 쪽인 사원의 기준급은 조인에 참여했어도 UPDATE 대상이 될 수 없습니다. 둘째, MERGE의 **WHEN MATCHED**와 **WHEN NOT MATCHED**는 둘 다 선택적이라 하나만 두어도 실행됩니다 — UPDATE만 하는 MERGE, INSERT만 하는 MERGE도 정상입니다. ③은 "비보존 칼럼도 수정 가능"·"두 절을 반드시 함께"라는 정반대 진술을 겹쳐 놓았습니다. 그래서 부적절한 것은 ③입니다.

①은 상여가 key-preserved가 되는 근거(사원.사원번호 PK)를, ②는 MERGE의 매칭·분기 동작을, ④는 유일성이 깨지면 UPDATE가 모호해져 거부되는 이유를 옳게 서술합니다.

선택지	판정	이유
①	○	사원.사원번호가 PK라 상여 한 행에 사원 한 행만 대응하므로 상여가 key-preserved입니다. 그 칼럼(상여금액)의 수정이 허용됩니다
②	○	MERGE는 한 문장으로 ON 조건 매칭 시 UPDATE, 미매칭 시 INSERT를 수행합니다. (나)의 구조 그대로입니다
③	×	비보존 테이블(사원)의 기준급은 수정 대상이 될 수 없고, MERGE의 두 WHEN 절은 각각 선택적이라 하나만 뒤도 실행됩니다 — 양쪽 다 반대 진술입니다
④	○	사원번호의 유일성이 깨지면 상여 행이 여러 행과 겹쳐 갱신값이 모호해지므로, key-preserved 조건 위반으로 UPDATE가 거부됩니다

▪ 이 문제의 핵심

1. 수정 가능 조인 뷰는 key-preserved 테이블의 칼럼만 수정할 수 있습니다. 비보존 쪽 칼럼은 조인에 참여해도 UPDATE 대상이 아닙니다.
2. key-preserved의 조건은 조인 후 유일성 보존입니다. 상대 테이블의 조인 키가 PK/유일해야 내 행이 한 번만 나타납니다.
3. MERGE의 WHEN MATCHED·WHEN NOT MATCHED는 둘 다 선택적입니다. 하나만 둔 MERGE(UPDATE만 또는 INSERT만)도 정상 실행됩니다.
4. 유일성이 깨지면 조인 뷰 UPDATE는 거부됩니다 — 한 행이 여러 행과 매칭돼 갱신값이 모호해지기 때문입니다.

- 한 줄 정리 조인 뷰에서는 key-preserved 테이블의 칼럼만 수정할 수 있고 MERGE의 두 WHEN 절은 각각 선택적인데, ③은 "비보존 칼럼도 수정 가능·두 절 반드시 함께"라 정반대로 서술해 부적절합니다.

문제 16. 정답 ①

Partition Pruning은 옵티마이저가 WHERE 조건의 값을 파티션 정의(LIST 값 목록)와 대조해 접근할 파티션을 미리 좁히는 최적화입니다. 이 대조가 가능하려면 조건이 파티션 키 칼럼(접수지역코드) 자체에 걸려 있어야 합니다.

- ② 접수지역코드 = '26' → 목록 대조 → p_busan 1개
 ④ 접수지역코드 IN ('11', '41') → 목록 대조 → p_seoul, p_gyeonggi 2개 (나머지 스킵)
 ① SUBSTR(접수지역코드, 1, 2)='11' → 키에 함수 → 목록과 대조 불가 → 전 파티션 스캔

①은 **SUBSTR(접수지역코드, 1, 2)**로 파티션 키에 함수를 씌웠습니다. 접수지역코드가 이미 2자리라 결과값이 원래 코드와 논리적으로 같더라도, 옵티마이저는 함수의 결과를 LIST 값 목록과 미리 대조할 수 없어 Pruning을 포기하고 **p_seoul ... p_etc** 모든 파티션을 스캔한 뒤 각 행에서 SUBSTR를 평가해 필터합니다. "앞 두 자리를 뽑았을 뿐이라 값이 같다"는 부분적 진실이 함정으로, 함수 한 줄이 최선(1개 파티션)을 최악(전 파티션 스캔)으로 뒤집습니다. 그래서 Pruning이 작동한다고 볼 수 없는 것은 ①입니다.

②④는 키를 가공하지 않은 조건이라 목록과 대조돼 필요한 파티션만 접근하고, ③은 파티셔닝 목적이 성능뿐 아니라 관리성·가용성까지 아우른다는 점을 옳게 설명합니다.

선택지	판정	이유
①	×	파티션 키에 SUBSTR 를 씌워 값 목록과 대조가 불가능해집니다. 값이 코드와 같아도 전 파티션을 스캔한 뒤 필터하므로 Pruning이 깨집니다
②	○	키를 가공하지 않은 등치 조건이라 '26'을 목록과 대조해 p_busan 한 파티션만 접근합니다. Pruning 작동합니다
③	○	파티셔닝 목적은 성능만이 아니라 파티션 단위 관리성과 장애 격리 가용성을 포함합니다. 표준적 서술입니다
④	○	두 리터럴을 각각 목록과 대조해 p_seoul·p_gyeonggi 두 파티션만 읽고 나머지는 건너뛰므로 Pruning이 작동합니다

▪ 이 문제의 핵심

1. Partition Pruning은 파티션 키에 직접 걸린 조건에서만 작동합니다. 키를 함수로 가공하면 파티션 정의와 대조할 수 없습니다.
2. LIST 파티셔닝은 값 목록과 등치·IN 조건을 대조해 해당 파티션만 남깁니다. 여러 값이면 여러 파티션이 남되 나머지는 건너뛰니다.
3. **SUBSTR·TO_CHAR** 등 키 가공은 결과가 원값과 같아 보여도 Pruning을 무력화합니다. "앞자리만 뽑았다"는 부분적 타당함이 함정입니다.
4. 파티셔닝의 목적은 성능·관리성·가용성입니다 — Pruning 성능만이 아니라 파티션 단위 관리와 장애 격리도 노립니다.
 - 한 줄 정리 키를 가공하지 않은 조건은 값 목록과 대조돼 Pruning되지만, **SUBSTR(접수지역코드,...)**처럼 키에 함수를 씌우면 결과가 같아 보여도 대조가 불가능해져 전 파티션을 스캔합니다.

문제 17. 정답 ②

분배 방식은 조인(Id 3)의 두 입력이 각각 어떤 경로로 조인 서버 집합에 도달하는지로 읽습니다.

Id 4 PX PARTITION HASH ALL (Q1,01, Pstart 1~Pstop 16) 운송기록: 파티션 단위 로컬 읽기, 재분배 없음
 Id 5 TABLE ACCESS FULL 운송기록 (Q1,01)
 Id 7 PX SEND PARTITION (KEY) :TQ10000 (Q1,00) 검수내역: 파티션 키에 맞춰 재분배(생산자)
 Id 9 TABLE ACCESS FULL 검수내역 (Q1,00)
 Id 3 HASH JOIN (Q1,01) ← Id 4(운송기록) · Id 6 PX RECEIVE(검수내역)를 받아 조인

운송기록(Id 5)은 이미 운송장번호 해시 16 파티션으로 나뉘어 있으므로, 조인 서버 집합 Q1,01이 Id 4 **PX PARTITION HASH ALL**로 각자 자기 파티션을 그대로 읽습니다. 위에 **PX SEND**가 없다는 것이 곧 재분배 없음의 표식입니다.

검수내역(Id 9)은 파티셔닝돼 있지 않으므로 그대로는 운송기록의 파티션과 맞물릴 수 없습니다. 그래서 생산자 집합 Q1,00이 Id 7 **PX SEND PARTITION (KEY)**로 검수내역을 운송기록의 파티션 경계에 맞춰 재분배하고, Id 6 **PX RECEIVE**가 Q1,01에서 받습니다.

즉 한쪽(운송기록)은 재분배 없이 파티션 로컬, 다른 쪽(검수내역)만 파티션 키로 재분배하는 이 방식이 부분(partial) 파티션 와이즈 조인입니다. 양쪽을 다 재분배하는 hash-hash와 달리 큰 테이블(운송기록 1억 9천만)을 네트워크로 흘리지 않아, 재분배량이 검수내역 한쪽으로 국한됩니다.

- ② partial PWJ : 운송기록 = SEND 없음(PARTITION HASH ALL), 검수내역 = SEND PARTITION(KEY) → Id 4 · Id 7과 일치
- ① full PWJ : 양쪽 다 SEND 없음(둘 다 같은 키로 파티셔닝) → 검수내역에 Id 7 SEND 있음
- ③ hash-hash : 양쪽 다 SEND HASH → 운송기록엔 SEND 자체가 없음
- ④ 역할 뒤바꿈 : 검수내역=로컬, 운송기록=SEND PARTITION(KEY) → Id 4 · Id 7과 반대(값 스왑)

PX SEND가 검수내역 쪽(Id 7)에만 있고 운송기록 쪽엔 없다는 사실이 full PWJ와 hash-hash를 함께 배제합니다. 그리고 재분배되는 쪽은 파티셔닝 안 된 검수내역이므로 ④처럼 역할을 뒤바꾼 진술도 틀립니다. 따라서 ②가 옳습니다.

선택지	판정	이유
①	×	검수내역은 파티셔닝돼 있지 않다고 발문이 못박았으므로 대응 파티션 쌍 자체가 없습니다. full 파티션 와이즈면 양쪽 조인 입력에 PX SEND 가 없어야 하는데 Id 7에 PX SEND PARTITION (KEY) 가 걸려 있고, Id 7은 QC로 넘기는 통로도 아닙니다 — QC 전송은 P->S인 Id 2 PX SEND QC (RANDOM) 입니다
②	○	운송기록은 Id 4 PX PARTITION HASH ALL 로 재분배 없이 파티션 로컬 읽기, 검수내역만 Id 7 PX SEND PARTITION (KEY) 로 운송기록 파티션에 맞춰 재분배 — 부분 파티션 와이즈 조인과 일치합니다
③	×	두 테이블 큐가 있다는 것은 사실이나 :TQ10001은 조인 결과를 QC로 보내는 Id 2용이지 분배용이 아닙니다. hash-hash면 Id 5·Id 9 위에 각각 PX SEND HASH 가 있어야 하는데 운송기록 쪽엔 PX SEND 가 아예 없고(Id 4는 PCWC 파티션 스캔), 검수내역 쪽도 SEND HASH 가 아니라 SEND PARTITION (KEY) 입니다
④	×	재분배되는 쪽과 로컬로 읽는 쪽을 뒤바꿨습니다. Id 7 PX SEND PARTITION (KEY) 가 걸린 것은 Id 8 PX BLOCK ITERATOR 로 읽히는 검수내역이고, Id 4 PX PARTITION HASH ALL 로 파티션 로컬 읽기를 하는 쪽이 운송기록입니다. 파티셔닝 안 된 검수내역을 로컬 기준으로 삼을 수도 없습니다

▪ 이 문제의 핵심

1. 부분(partial) 파티션 와이즈 조인은 한쪽만 재분배합니다. 이미 조인 키로 파티셔닝된 운송기록은 재분배 없이 파티션 로컬로 읽고, 안 나뉜 검수내역만 그 경계에 맞춰 재분배합니다.
2. 재분배 여부는 **PX SEND** 유무로 판별합니다. 운송기록 위엔 SEND가 없고(Id 4 PX PARTITION HASH ALL), 검수내역 위엔 Id 7 **PX SEND PARTITION (KEY)**가 있습니다.
3. **SEND PARTITION (KEY)**는 상대 테이블의 파티션 경계에 맞추는 재분배입니다. 조인 키 해시로 새 버킷을 만드는 hash-hash의 **SEND HASH**와 다릅니다.

4. 어느 쪽이 재분배되는지가 판별의 핵심입니다. 재분배되는 쪽은 파티셔닝 안 된 검수내역이며, 이를 운송기록과 뒤바꾸면 틀린 진술이 됩니다.

▪ 한 줄 정리 운송기록은 Id 4 **PX PARTITION HASH ALL**로 재분배 없이 파티션 로컬로 읽고 검수내역만 Id 7 **PX SEND PARTITION (KEY)**로 운송기록 파티션에 맞춰 재분배하므로, 큰 테이블을 흘리지 않는 부분 파티션 와이즈 조인입니다.

문제 18. 정답 ②

SELECT 절의 사용자 정의 함수는 결과 행 수만큼 반복 호출됩니다. 그리고 함수를 한 번 호출할 때마다 실행 엔진은 SQL 엔진과 PL/SQL 엔진 사이를 오갑니다(컨텍스트 스위치). 여기에 더해 함수 본문이 다시 SQL을 수행하면, 그 내부 SQL이 재귀 호출(recursive call)로 매 행마다 반복 실행되어 부하가 이중으로 쌓입니다.

그래서 처방은 호출 횟수 자체를 줄이는 것입니다. 함수가 하는 조회를 조인으로 풀어 SQL 한 문장 안에서 처리하거나, 입력값의 종류가 적다면 스칼라 서브쿼리로 감싸 캐싱을 노력 반복 호출을 줄입니다. ②가 원인(행마다 컨텍스트 스위치 + 재귀 SQL)과 처방(조인·스칼라 서브쿼리 대체)을 정확히 짚지었습니다.

나머지는 일부 경우에만 성립하는 성질을 전제로 부풀린 하위항목 전체화 오류입니다.

①: 커서를 열 때 함수가 준비된다는 것과 어느 엔진이 본문을 실행하느냐는 별개입니다. PL/SQL 함수 본문은 SQL 엔진이 아니라 PL/SQL 엔진으로 넘어가 수행되므로 호출마다 경계를 넘고, 늘어나는 것은 본문 연산량만이 아니라 호출 횟수에 비례하는 스위치 비용입니다.

③: 컨텍스트 스위치는 SQL↔PL/SQL 경계를 넘는 호출 그 자체에서 발생합니다. 함수가 내부에서 SQL을 수행할 때 생기는 재귀 호출 부하는 그 위에 추가로 얹히는 별개 비용일 뿐입니다. 그래서 순수 계산 함수도 행마다 스위치를 물고, 조인으로 녹여 호출을 없애면 그만큼 이득이 납니다. "내부 SQL이 있을 때만 스위치가 생긴다"는 한 갈래를 전제로 부풀린 오류입니다.

④: 스칼라 서브쿼리 캐싱은 입력값을 키로 결과를 재사용하는 최적화와 입력값이 반복될 때(값의 종류가 적을 때)만 호출을 줄입니다. 입력이 행마다 제각각이면 캐시 적중이 없어 호출이 한 번으로 줄지 않고, 해시 충돌 시에는 오히려 재수행까지 겹칩니다. 캐싱의 제한적 효과를 입력과 무관한 성질로 일반화한 오류입니다.

선택지	판정	이유
①	×	커서를 열 때 함수가 준비되는 것과 본문을 어느 엔진이 실행하느냐는 별개입니다. 본문은 PL/SQL 엔진으로 넘어가 수행되므로 호출마다 경계를 넘고, 행이 늘면 본문 연산량뿐 아니라 호출 횟수에 비례하는 스위치 비용도 함께 늘어납니다
②	○	행마다 호출되어 컨텍스트 스위치와 재귀 SQL이 겹쳐 부하가 커지므로, 조인·스칼라 서브쿼리로 호출을 줄이는 것이 효과적입니다
③	×	컨텍스트 스위치는 SQL↔PL/SQL 경계를 넘는 호출 자체에서 생기며 내부 SQL 유무와 무관합니다. 재귀 호출은 그 위에 얹히는 별개 비용이므로, 순수 계산 함수도 행마다 스위치를 물고 조인으로 녹여 호출을 없애는 실익이 그대로 남습니다
④	×	스칼라 서브쿼리 캐싱은 입력값을 키로 결과를 재사용하므로 값의 종류가 적을 때만 호출을 줄입니다. 입력이 행마다 제각각이면 적중이 없어 호출이 한 번으로 줄지 않고, 컨텍스트 스위치도 그대로 남습니다

▪ 이 문제의 핵심

1. **SELECT** 절 UDF는 행마다 호출되고, 호출마다 SQL↔PL/SQL 컨텍스트 스위치가 발생합니다.
2. 함수 본문의 SQL은 재귀 호출로 반복 실행되어, 컨텍스트 스위치 위에 부하가 이중으로 쌓입니다.
3. 컨텍스트 스위치는 호출 그 자체의 비용입니다 — 내부 SQL이 없어도 발생하며, 재귀 SQL은 별개로 추가되는 비용입니다.
4. 처방은 호출 횟수 줄이기. 조인으로 SQL 한 문장에 녹이거나, 입력 종류가 적을 때 스칼라 서브쿼리 캐싱을 활용합니다.

▪ 한 줄 정리 **SELECT** 절 UDF는 행마다 컨텍스트 스위치와 재귀 SQL을 유발하므로 조인·스칼라 서브쿼리로 호출을 줄이는 것이 효과적이며, "내부 SQL이 있을 때만 스위치가 생긴다"거나 "캐싱이 입력과 무관하게 호출을 한 번으로 줄인다"는 것은 일부 성질을 전제로 부풀린 오류입니다.

문제 19. 정답 ①

관건은 스냅샷의 기준 시점이 트랜잭션 시작이냐 문장 시작이냐입니다. 방식별로 (B)를 계산하면 SNAPSHOT만 4200이고 나머지는 4650으로 같습니다.

[읽는 값 - 초기 4200, TX2가 t3에 4650으로 커밋]

	(A) t2 첫 읽기	(B) t4 둘째 읽기
SQL Server SNAPSHOT	4200	4200 ← 트랜잭션 시작 스냅샷(반복 가능)
SQL Server 기본 RC(잠금)	4200	4650 ← 문장마다 커밋된 최신값
SQL Server RCSI(문장 스냅샷)	4200	4650 ← 문장 시작 스냅샷

SNAPSHOT : (A)=(B)=4200 (유일하게 반복 가능)
그 외 : (A)=4200, (B)=4650 (비반복 읽기)

SQL Server SNAPSHOT 격리는 트랜잭션 시작 시점의 행 버전을 트랜잭션 내내 봅니다. t3에서 TX2가 4650으로 커밋해도 TX1의 (B)는 여전히 스냅샷의 4200 — 두 읽기가 반복 가능합니다.

RCSI와 오라클 기본 READ COMMITTED는 모두 문장 시작 기준입니다. 두 번째 SELECT는 새 문장이므로 그 시점에 커밋되어 있는 4650을 읽어 (B)=4650이 됩니다. 오라클 MVCC의 읽기 일관성 단위도 트랜잭션이 아니라 문장입니다.

①은 (A)=(B)=4200(SNAPSHOT)과, 문장수준 방식이었다면 (B)=4650이 된다는 대비를 모두 맞혔습니다.

선택지	판정	이유
①	○	SNAPSHOT은 트랜잭션 스냅샷이라 (A)=(B)=4200으로 반복 가능하고, 문장수준(오라클 RC·SQL Server 기본 RC)이면 (B)=4650으로 달라집니다
②	×	(A)=4200과 t3의 4650 커밋까지는 맞지만 결론이 반대입니다. SNAPSHOT은 t1 BEGIN TRAN 시점 스냅샷을 트랜잭션 내내 보므로 t4의 (B)도 4200이며, 비반복 읽기를 허용하는 것이 아니라 오히려 막습니다. (B)=4650은 문장수준 방식의 값을 끌어다 붙인 값스압입니다
③	×	RCSI의 스냅샷 기준은 트랜잭션이 아니라 문장 시작입니다. t4의 SELECT는 새 문장이라 그 시점에 커밋돼 있는 4650을 읽어 (B)=4650이 되고, SNAPSHOT과 결과가 같습니다. 두 옵션은 둘 다 버전 저장소를 쓰며 차이는 바로 그 기준 시점이므로, (B)=4200은 SNAPSHOT의 값과 뒤바뀐 것입니다
④	×	오라클도 연두로 읽기 일관성을 보장한다는 앞부분은 맞지만 그 단위는 트랜잭션이 아니라 문장입니다. t4의 SELECT는 자기 문장 시작 시점의 SCN을 기준으로 삼아 이미 커밋된 4650을 읽으므로 (B)=4650이며, (B)=4200은 SQL Server SNAPSHOT의 값과 맞바꾼 서술입니다

▪ 이 문제의 핵심

1. SQL Server SNAPSHOT 격리는 트랜잭션 시작 시점 스냅샷을 봅니다. 중간에 커밋된 갱신을 무시해 (A)=(B)=4200으로 반복 가능합니다.
2. RCSI는 문장 시작 스냅샷입니다. 새 문장인 두 번째 SELECT는 커밋된 최신값 4650을 읽으므로 SNAPSHOT과 결과가 다릅니다.
3. 오라클 기본 읽기 일관성의 단위도 문장입니다. 트랜잭션 전체가 아니라 문장 시작 시점 기준이라 (B)=4650이 됩니다.
4. 트랜잭션 스냅샷과 문장 스냅샷은 (B)의 값(4200 vs 4650)으로 같습니다 — 어느 시점을 기준으로 스냅샷을 뜨느냐가 반복 가능성을 결정합니다.

▪ 한 줄 정리 SNAPSHOT 격리만 트랜잭션 시작 스냅샷이라 (A)=(B)=4200으로 반복 가능하고, RCSI·오라클 기본 RC 같은 문장수준 일관성은 두 번째 읽기에서 커밋된 4650을 보아 (B)가 달라집니다.

문제 20. 정답 ③

TM(테이블) 락은 오브젝트 단위 락이고, 모드 사이의 호환성이 대기 여부를 결정합니다. 표를 읽으면 S(4) 보유와 RX(3) 요청이 충돌하는 그림이 드러납니다.

SID 41 : TM 54021 LMODE=4(S) BLOCK=1 → LOCK TABLE IN SHARE MODE 로 테이블 S 보유
SID 55 : TM 54021 REQUEST=3(RX) → UPDATE가 RX 요청 → S와 비호환 → 대기

[TM 호환성] S(4) vs RX(3) = 비호환
S(4) vs S(4) = 호환 (여러 세션이 함께 SHARE 가능)

③은 호환성을 정반대로 뒤집었습니다. 오라클 TM 락에서 S(4)와 RX(3)는 비호환입니다 — SHARE 모드로 테이블을 잠근 동안에는 다른 세션의 DML(RX 요청)이 막힙니다. 표의 세션 55가 **REQUEST=3**으로 대기 중인 것이 바로 그 증거입니다. 그런데 ③은 "S와 RX가 호환이라 UPDATE가 대기 없이 진행된다"고 했으니, 표가 보여 주는 대기 상태와 정면으로 어긋납니다. 그래서 부적절한 것은 ③입니다.

①은 41이 보유한 것이 행 단위 TX가 아닌 테이블 수준 TM(S 모드)임을, ②는 55의 UPDATE가 RX를 요청하다 비호환으로 대기함을, ④는 S끼리는 호환이지만 RX 요청은 여전히 막힘을 옳게 설명합니다.

선택지	판정	이유
①	○	41의 락은 ID1=54021 오브젝트에 걸린 S(4) 모드 TM 락으로, 행 단위 TX 락이 아니라 테이블 수준 락입니다. BLOCK=1로 상대를 막고 있습니다
②	○	UPDATE는 갱신 전 테이블에 RX(3)를 요청하는데 S(4)와 비호환이라 얻지 못하고 REQUEST=3으로 대기합니다. 표의 두 번째 행 그대로입니다
③	×	S(4)와 RX(3)는 비호환입니다. SHARE 락이 걸린 동안 DML의 RX 요청은 막혀 대기하는데, ③은 호환이라 즉시 진행한다고 반대로 서술했습니다
④	○	S(4)끼리는 호환이라 62가 41과 함께 SHARE를 보유해 대기하지 않지만, RX(3)는 S와 비호환이라 55의 요청은 그대로 막힙니다

▪ 이 문제의 핵심

1. TM 락은 테이블(오브젝트) 수준 락입니다. 행을 식별하는 TX 락과 층위가 다르며, **LOCK TABLE**·DDL·DML이 각기 다른 TM 모드를 겁니다.
 2. **LOCK TABLE ... IN SHARE MODE**는 S(4) 모드 TM 락을 겁니다. 조회는 허용하되 다른 세션의 DML(RX 요청)은 막습니다.
 3. S(4)와 RX(3)는 비호환입니다. SHARE가 걸린 동안 UPDATE·INSERT·DELETE는 RX를 얻지 못해 대기합니다.
 4. S(4)끼리는 호환이라 여러 세션이 함께 SHARE로 잠글 수 있지만, 그 사이에도 RX 요청은 계속 막힙니다.
- 한 줄 정리 **LOCK TABLE ... IN SHARE MODE**가 건 S(4) TM 락은 DML의 RX(3) 요청과 비호환이라 세션 55가 대기하는데, "S와 RX가 호환이라 즉시 갱신한다"는 ③은 호환성을 정반대로 뒤집어 부적절합니다.